

Project Report Template

Advanced Analytics and Applications

Team 07



Authors:

Larissa Theilmann (7432976)

Emre Uyar (7443638)

Tjorge Orlitz (7443662)

Nils Hornstein (7369566)

Contact: torlitz@smail.uni-koeln.de

Supervisor: Univ.-Prof. Dr. Wolfgang Ketter

Co-Supervisor: Janik Muires

Chair of Information Systems for Sustainable Society
Faculty of Management, Economics and Social Sciences
University of Cologne

2026-04-01

Table of contents

1	Introduction	1
1.1	Motivation and Problem Statement	1
1.2	Research Questions	1
2	Data Description	2
2.1	Data Sources	2
3	Data Preparation	2
3.1	Taxi Trip Description and Cleaning	2
3.2	Weather Data	3
3.3	Points of Interest	3
3.4	Spatial and Temporal Discretisation	4
4	Data Analytics	4
4.1	Descriptive (Spatial) Analytics	4
4.1.1	Spatial Patterns	4
4.1.2	Temporal Patterns	5
4.1.3	Effect of Spatio-Temporal Resolution	5
4.1.4	Demand Hot Spots	5
4.2	Predictive Analytics (SVM & Neural Networks)	5
4.2.1	Support Vector Machine	6
4.2.2	Deep Learning Model	7
4.2.3	Model Comparison	8
4.2.4	Shortfalls and Improvement Levers	8
4.3	Smart Charging (Reinforcement Learning)	9
4.3.1	Executive Summary	9
4.3.2	Problem Framing & Methodology	9
4.3.3	Results	10
5	Discussion and Outlook	10
5.1	Summary of Findings & Implications for the Client	10
5.2	Limitations & Future Work	11
A	Additional Figures and Tables	11
A.1	Taxi Data Preparation (1.1)	11
A.2	Weather Data Preparation (1.2)	11
A.3	Points of Interest Data Preparation (1.3)	16
A.4	Temporal-Spatial Discretization (1.4)	16
A.5	Descriptive Analysis: Spatial (2.1)	16
A.6	Descriptive Analysis: Temporal (2.2)	16
A.7	Descriptive Analytics: Weather (2.4)	16
A.8	Feature Engineering (3.1)	16
A.9	SVR Hyperparameter Search Grid (3.2)	17
A.10	SVR: Direct Model (3.2a)	18
A.11	SVR: Residual-on-Baseline Model (3.2b)	18
A.12	Deep Learning Prediction (3.3)	19
A.13	SVM vs. Neural Network Performance Comparison (3.4)	19
A.14	Reinforcement Learning (4.1)	19
	References	36

List of Figures

1	Top 30 most frequent raw OSM POI types in Chicago, before category mapping.	12
2	Top 50 most frequent POI types remaining after category filtering (unmapped ‘other’ types dropped).	12
3	Total POI count per H3 hexagon (resolution 7).	13
4	POI count per H3 hexagon (resolution 7), faceted by category.	14
5	Spearman correlation of POI category counts across H3 hexagons (resolution 7).	15
6	SVR skill against the historical-mean baseline, by spatial and temporal granularity.	20
7	Left: what the fitted model leans on (grouped permutation importance, as each block’s share of the level’s total). Right: what a retrained model actually needs (drop-column ablation, % increase in MAE when the block is removed). The two disagree sharply on base_demand — the fitted model depends on it almost entirely, yet a model refitted without it loses very little, because lat/lon/hour/day-of-week can reconstruct it.	20
8	Residual-on-baseline SVR skill against the historical-mean baseline, by spatial and temporal granularity.	21
9	Direct (3.2a) vs. residual-on-baseline (this notebook) SVR skill, both independently tuned, per level.	21
10	Actual vs. predicted demand per temporal resolution.	22
11	MAE per hexagon per temporal resolution.	23
12	Relative error per hexagon per temporal resolution.	24
13	R ² per hexagon per temporal resolution.	25
14	MAE_NN - MAE_SVM per hexagon per temporal resolution. Blue = SVM wins, red = FNN wins.	26
15	Test MAE by day of week per temporal resolution.	27
16	Test relative error (MAE / mean demand) by day of week per temporal resolution.	28
17	Test MAE by hour of day (h3_7/hourly).	29
18	Test relative error (MAE / mean demand) by hour of day (h3_7/hourly).	29
19	City-wide demand over the test period per temporal resolution.	30
20	Test relative error (MAE / mean demand) by segment per temporal resolution.	32
21	Figure 1a: Training score over time.	32
22	Figure 1b. Probability of running out of energy over time.	33
23	Figure 2. Learned charging policy. The color indicates the chosen charging speed for each battery-level/time combination.	33
24	Figure 3a. DQN training score.	34
25	Figure 3b. DQN failure rate during training.	34
26	Figure 4. Best DQN policy found across the five training runs. The Q-table stores entries only for states that were actually visited during training (visit_counts > 0) and thus, combinations of time step and battery level that were never reached remain empty. The DQN network, by contrast, is a continuous function that produces an output for any given input. This includes states that never occurred during training (e.g. battery=60 at t=0). Thereby, both plots differ in terms of their appearance.	35

1 Introduction

1.1 Motivation and Problem Statement

The ride-hailing market is already crowded. Uber, Lyft, FreeNow, Lime, and Tier compete for the same trips, and current market reports point to ride-hailing, increasingly paired with autonomous and electric vehicles, as the segment that will dominate urban mobility in the near term. Our client, a well-established German car manufacturer, is building a ride-hailing platform on a fully electrified vehicle fleet, wanting to launch first in the United States, where customers have proven more receptive to novel mobility business models.

The client can design and build vehicles, but has no experience running a ride-hailing fleet. Managing an electric fleet adds a further constraint: Charging schedules directly affect vehicle availability in ways a conventional fleet does not face. To plan routes, size a fleet, and decide where to deploy vehicles, the client needs a clear picture of how demand moves through a city across space and time.

This report builds from historical taxi trip data for Chicago, made available by the Chicago Data Portal and covering trips from 2024 onward. Taxis are not ride-hailing vehicles, but in a market the client has not yet entered, they are the closest available proxy for one: metered urban trips, dispatched on demand, subject to the same weather and time-of-day patterns that will govern the client's own fleet. We supplement the trip records with independently collected weather data and, where it adds value, point-of-interest data from OpenStreetMap, and we analyze demand at spatial resolutions ranging from coarse census tracts to fine-grained H3 hexagon grids, and at multiple temporal resolutions, to establish which level of detail the data can actually support. We also investigate how a vehicle should charge at a driver's home between shifts, given a battery of finite capacity and a next-shift energy requirement that is not known with certainty in advance, using Reinforcement Learning.

1.2 Research Questions

The analysis that follows is organized around five questions, each feeding into the next.

1. How does taxi demand vary across space and time in Chicago, and what plausibly explains the patterns observed at different spatial units and temporal bin sizes?
2. Where are the persistent spatial hot spots of demand, and what do Gaussian mixture models and kernel density estimation reveal about their structure?
3. Can demand be predicted for a specific spatial unit and time bucket rather than simply extrapolated from the previous hour, and how does a support vector machine compare against a feedforward neural network on this task?
4. What charging policy should an electric taxi driver follow at home, given a charging rate that can be adjusted every fifteen minutes, a battery capacity that cannot be exceeded, and a next-shift energy requirement that is only known probabilistically at the moment the driver departs?
5. What do these findings imply for the client's fleet operations, and in particular, should the client invest in private home charging infrastructure or rely on public charging networks instead?

2 Data Description

2.1 Data Sources

Three raw datasets underpin the analysis: taxi trip records, hourly weather observations, points of interest, accompanied by a boundary layer of Chicago’s community areas. Each is described below by origin, coverage, and native granularity; the cleaning and feature-engineering steps applied to each follow in Section 3.

The taxi trip records come from the Chicago Data Portal’s Taxi Trips dataset, which publishes one row per dispatched, metered taxi ride in the city. The raw download spans January 1, 2024 through May 31, 2026 and holds 16,086,255 trips across 23 fields: trip and taxi identifiers, start and end timestamps, duration and distance, the fare broken into its components and as a total, the operating company, and pickup and dropoff locations given as census tract, community area, and centroid coordinates. Locations are masked for privacy, so a published coordinate is the centroid of the containing census tract rather than the true pickup point, and the tract itself is withheld on just over half of all trips (55.3% of pickups, 56.6% of dropoffs) against 2.8% and 8.8% for community area. Each trip is a ride requested wherever and whenever a passenger wanted one, which is the demand pattern the client’s own fleet will have to anticipate.

Weather observations come from the ERA5-Land reanalysis product, pulled through the Copernicus Climate Data Store API for a single coordinate near Chicago (latitude 42, longitude -87.8) rather than a network of ground stations. The record runs from January 1, 2024 through May 12, 2026 at hourly resolution and carries six variables: 2-metre air temperature, total precipitation, snow cover, snow depth, and the two horizontal components of wind at 10 metres. A reanalysis combines numerical weather prediction models with observational data through data assimilation, resulting in a temporally continuous representation of atmospheric conditions, which no single Chicago weather station could promise.

Points of interest come from OpenStreetMap, extracted with OSMnx for the “Chicago, Illinois, USA” boundary and limited to three tag families: amenity, shop, and tourism. The raw download returns 43,237 points. Unlike the trip and weather data, this is a snapshot of the map as it stands today, not a historical record tied to the trip period. The boundary layer for Chicago’s 77 official community areas is used for spatial reference and choropleth mapping throughout the report. It is a distinct unit from the census tracts already present in the trip data itself.

3 Data Preparation

This section describes the preparation pipeline that turns the raw sources described in Section 2 into the analysis-ready datasets used throughout the report. A detailed table of the engineered features can be seen in the Table 2 in the appendix.

3.1 Taxi Trip Description and Cleaning

Cleaning the raw trip table (Section 2) proceeds in three stages: trimming columns, removing unusable rows, and splitting the result into the two parallel datasets the rest of the pipeline consumes.

Seven of the 23 raw fields are dropped as redundant. The individual fare components (fare, tips, tolls, and extras) are already summed into the trip total, the payment type says nothing about where or when demand appears, and the two point-geometry columns merely repeat the latitude and longitude stored separately. Sixteen fields remain: trip and taxi identifiers, start

and end timestamps, duration and distance, pickup and dropoff census tract and community area codes, the fare total, the operating company, and pickup and dropoff coordinates. The export writes decimals with commas and prefixes currency with a dollar sign, so these columns are parsed locale-aware before any numeric comparison is made on them. From the coordinates we also derive an H3 cell index per trip at resolutions 7, 8, and 9, which gives the hexagonal units used alongside tracts and community areas later.

Rows are removed on two grounds. Records missing a taxi identifier, an end timestamp, a duration, a distance, or a fare total are dropped outright; each of these is absent on well under 0.2% of trips, a share small enough that imputation would add assumptions without adding information. A sanity filter then removes trips with a non-positive duration or distance, or a negative total, none of which describe a journey that actually happened. No statistical outlier removal is applied beyond that: extreme but plausible values, such as long airport transfers, are legitimate demand and are deliberately kept.

Because census tract is withheld far more often than community area, the cleaned data is kept as two parallel datasets rather than forcing one compromise on every downstream analysis. The larger retains every trip with a known community area; the stricter keeps only those that also carry a census tract. Both are truncated to the last hour the weather series covers, so that no model can later be trained or evaluated against forward-filled weather. That leaves **12,949,755 trips at community-area completeness** and **6,052,512 at census-tract completeness**, both spanning January 1, 2024 through May 12, 2026.

3.2 Weather Data

The final dataset comprises 17,544 hourly observations across nine columns (a timestamp, six raw variables (Table 1), and two engineered wind variables) with no missing values, consistent with the reanalysis nature of the data. A small number of precipitation and snow cover values were marginally negative due to rounding errors in the underlying computations and were clipped to zero; temperature and precipitation were additionally converted to Celsius and millimeters for interpretability. Exploratory analysis confirms clear seasonal patterns, with snow cover and snow depth peaking between November and March/April and falling to near zero in summer. Snow cover approaches 100% precisely when snow depth peaks and temperatures are lowest, confirming internal consistency, while precipitation shows no strong correlation with snow variables, as expected given their distinct physical origins. No anomalies or implausible extremes were found. The temperature ranges from below -30°C to roughly 35°C , precipitation stays below ~ 20 mm/hour, and snow depth does not exceed 30 cm which is consistent with Chicago’s historical climate. For the client, this means the weather inputs for our models are trustworthy and reliable to use.

3.3 Points of Interest

Point-of-interest data was optional under the assignment brief, but there is precedent for treating it as a demand signal: Yan et al. (2020) predict taxi demand at city scale by combining several sources of urban geospatial data, point-of-interest records among them. We therefore include POI density to test how much it explains about where and when ride demand originates in Chicago. Coverage across the three tag families is uneven: 7,053 records lack an amenity tag, 37,552 lack a shop tag, and 41,738 lack a tourism tag, so most points carry only one of the three. We collapsed them into a single `poi_type` field by taking amenity where present and falling back to shop, then tourism, which left every point with one of 311 distinct raw types and none without a type or a geometry.

At 311 types the data is too granular to serve directly as model features, so we mapped each raw type onto one of fourteen broader categories that plausibly drive ride-hailing demand: food and drink, nightlife, entertainment, leisure and sports, grocery, shopping, health, finance, education, lodging, transport, automotive, services, and civic and community. The mapping doubles as a filter. Fifty-five raw types like infrastructure and street furniture (benches, parking entrances, post boxes, waste bins), fit no category and were dropped, leaving 20,108 categorised points of interest. Roughly half of those are building polygons rather than single points, so each was reduced to its centroid and assigned to a spatial unit, then aggregated to all three units used downstream: H3 resolution 7, census tract, and community area. The handful of points that fall just outside the official city boundary are excluded from the tract and community aggregations but retained at the H3 level, which is defined continuously over any coordinate.

3.4 Spatial and Temporal Discretisation

The grid is built from H3 resolution 7, every hexagon whose centroid falls inside the Chicago boundary. Census tract and community area join the same lookup by matching each hexagon centroid against the tract and community polygons, giving one crosswalk across all three spatial units. Every unit is then crossed with two temporal bins, hourly and daily, with pickups aggregated against all spatial and temporal levels. Spatial units that never register a single pickup over the full 2.4-year window are dropped before the grid is written; the reduction removes only all-zero rows and costs no trips.

4 Data Analytics

This section covers all analytical methods applied to the prepared data: descriptive spatial analytics, predictive modelling of demand (SVM and neural networks), and reinforcement learning for smart charging.

4.1 Descriptive (Spatial) Analytics

To better understand the characteristics of taxi demand before developing predictive models, we first perform a descriptive analysis of the dataset. The analysis investigates spatial and temporal demand patterns, examines trip characteristics, compares different spatial aggregation levels, and identifies persistent demand hot spots. **### Demand Patterns** The taxi demand data exhibit strong spatial and temporal heterogeneity. Most trips are relatively short in terms of both travel distance and duration, while trip fares show a pronounced right-skewed distribution. Consequently, the majority of journeys are inexpensive, whereas a comparatively small number of long-distance trips account for substantially higher fares. Spatially, taxi activity is highly concentrated in a limited number of locations. Pickup and dropoff demand are strongly correlated, indicating that areas generating many departures also receive many arrivals. However, comparing pickup and dropoff distributions reveals localized differences that likely reflect commuting flows, airport traffic, and the separation of residential and commercial districts. Additional descriptive statistics and distribution plots are provided in **Appendix A**.

4.1.1 Spatial Patterns

Taxi demand was analysed using community areas, census tracts and H3 hexagonal grids at multiple resolutions. Although all representations reveal similar large-scale demand structures, the selected spatial resolution considerably influences the observed level of detail. Community areas provide a coarse overview of demand, while census tracts capture substantially finer local

variation. H3 resolution 7 offers a compromise between spatial detail and robustness, whereas H3 resolution 8 largely reproduces the census tract representation. This behaviour results from the privacy-preserving coordinate masking applied to the original dataset, where published coordinates correspond to census tract centroids whenever available. **[Figure X: Spatial demand distribution]** Across all spatial resolutions, demand remains highly concentrated in only a small number of regions. Increasing spatial resolution improves the visibility of localized demand patterns but simultaneously introduces considerable sparsity, with many spatial units containing few or no observations during large parts of the observation period. Further spatial comparisons, concentration measures, Lorenz curves and pickup-dropoff difference maps are shown in **Appendix A**.

4.1.2 Temporal Patterns

Taxi demand follows clear and recurring temporal patterns. Demand is lowest during the early morning hours, increases throughout the day and reaches pronounced peaks during commuting and evening leisure periods. Distinct differences are also observed between weekdays and weekends, reflecting changes in travel behaviour. **[Figure X: Average hourly taxi demand]** Besides the intraday cycle, demand also varies systematically across weekdays and seasons, indicating that temporal information is highly informative for subsequent forecasting models. Additional temporal analyses, including weekday-weekend comparisons and seasonal demand patterns, are provided in **Appendix A**.

4.1.3 Effect of Spatio-Temporal Resolution

Abstimmen mit predictive

4.1.4 Demand Hot Spots

Persistent demand hot spots were identified using Gaussian Mixture Models (GMM) and Kernel Density Estimation (KDE). Both approaches consistently identify the central business district and major transportation hubs as the dominant centres of taxi activity. **[Figure X: Gaussian Mixture Model hot spots]** The probabilistic clustering produced by the GMM groups regions with similar spatial demand characteristics, while KDE provides a continuous representation of demand intensity. Both methods highlight areas associated with dense commercial activity, transportation infrastructure and entertainment facilities, demonstrating the strong influence of land use on taxi demand. The complementary KDE visualisation is presented in **Appendix A**.

4.2 Predictive Analytics (SVM & Neural Networks)

Fleet planning needs to know where and when demand will appear before a shift starts. We predict trip starts per spatial unit and time bucket from the cell's identity and location, calendar, weather, and POI features alone, a scenario question such as expected demand on a rainy Sunday morning on the outskirts, not an extrapolation from the previous hour. Two model families are compared: a Support Vector Machine, fitted directly on demand and on the residual over a historical-mean baseline, and a feedforward neural network. Both train across six grids crossing three spatial units (H3 resolution 7, census tract, community area) with hourly and daily bins, and both are scored against the same historical-mean baseline, letting us ask whether a learned model, and a deep one especially, earns its added complexity.

The target is `Total_Trip_Start`, the trip count per spatial unit and time bucket. The three spatial units carry 108 active hexagons at H3 resolution 7, 496 active census tracts, and 77 community areas, after dropping inactive units with no pickups in the observation window during grid construction (Section 3). Sparsity differs sharply: zero-demand rows range from 0.8% at community/daily to 96.5% at census/hourly, driving the sampling decisions taken separately for each model family below.

The reference point throughout is a historical mean per spatial unit, keyed by month and day of week, plus hour on the hourly grids. Computed on the training split alone and merged into validation and test, it carries no future information and serves double duty: as the `base_demand` feature available to both models, and as the skill-score benchmark, $1 - \text{MAE_model} / \text{MAE_baseline}$. Skill rather than raw MAE is reported because mean demand differs by roughly a factor of 80 across the six grids, making raw errors incomparable between them.

Train, validation, and test are cut along the calendar rather than drawn at random: the earliest 50% of unique dates train the models (2024-01-01 to 2025-03-06), the next 20% validate (2025-03-07 to 2025-08-26), and the final 30% are held out for testing (2025-08-27 to 2026-05-12). A random row-level split would leak autocorrelated demand into the test set and contaminate `base_demand`. Model selection runs on validation; test is scored once, for the selected models.

4.2.1 Support Vector Machine

Two SVR models train on identical splits, samples, and candidate feature sets, differing only in the prediction target. The direct model regresses $\log_{10}(\text{Total_Trip_Start})$ and converts predictions back with `expm1`, clipped at zero. The residual model instead predicts the log-space correction to the historical mean, $\log_{10}(\text{demand}) - \log_{10}(\text{base_demand})$, and reconstructs counts as `expm1(log10(base_demand) + correction_hat)`; it never sees the raw target, only how far a cell and time bucket departs from its own history. The reasoning: `base_demand` already carries much of the predictable signal, so a model reproducing it from scratch would spend capacity re-deriving what the pipeline already computed. Because the correction sits on a far smaller scale and variance than $\log_{10}(\text{demand})$, hyperparameters tuned for one target do not transfer to the other, so each is tuned separately.

SVR's fit cost is quadratic in sample size, so each level trains on a class-balanced sample: all non-zero training rows plus an equal number of zero rows (4,000 for the kernel scan, 8,000 for the grid search, the winner then re-scored on a 50,000-row validation draw). Levels with little zero demand, such as community/daily at 0.8%, end up non-zero-heavy instead. A uniform sample of the same size would be 94% empty at h3_7/hourly and 96.5% at census/hourly, leaving little for a model to learn beyond predicting zero. Validation and test stay unbalanced, so both models are scored at a natural prevalence they never trained on.

Tuning runs in two stages per level and feature set: a kernel scan compares linear, polynomial, and RBF at a fixed baseline setting, judged by validation R^2 in count space, and the winning kernel alone carries into a three-fold grid search over `C` and `gamma` (extended by `degree` where polynomial wins), scored by count-space MAE. Four feature sets compete at every level: a basic calendar/location/history block, plus weather, plus POI, plus both. Table 3 lists every searched value and sample size; Table 4 lists the four feature blocks.

4.2.1.1 Validation Results

The direct model does not beat the historical mean feature it is given. At five of six levels its tuned winner trails the baseline on validation MAE, from -3.5% at community/daily to -35.5% at census/hourly, and the one level it wins, h3_7/daily, it wins by just 0.2%. Predicting the

correction instead lifts the skill score at all six levels, though unevenly: community/daily is the one clear win, at +5.3% ($R^2 = 0.970$); h3_7/daily turns its tie into a slim +1.2%; h3_7/hourly and census/daily stay near ties at +0.3%; community/hourly remains just short of the baseline at -1.0%. Census/hourly, the sparsest grid at 96.5% zero rows, loses heavily under both strategies (-35.5% direct, -32.1% residual), with MAE skill deeply negative while RMSE skill stays roughly flat (+0.7% residual). Daily aggregation beats hourly at all three spatial resolutions under both strategies, since hourly bins are sparser and noisier.

Every winning direct model needs the flexible RBF kernel; four of the six winning residual models do just as well with a straight line (both h3_7 levels, census/hourly, and community/daily). Both models operate on a compressed scale that gets expanded back into trip counts, so a small error before expansion becomes large after it. The direct model predicts demand itself, anywhere from zero to several thousand trips a day, and needs the extra curvature to stop those errors from compounding; the residual model predicts only a small adjustment, so even a straight line stays close enough that expansion cannot run away with it. All six residual winners settle at $C=1$, against direct winners spread from $C=1$ to $C=50$.

The two best models are both residual. At community/daily, a linear fit at $C=1$ on the basic+weather feature set brings the average error down from the baseline’s 36.19 trips per area-day to 34.25, at $R^2 = 0.970$. Second is h3_7/daily, on the basic+poi feature set, also linear at $C=1$, moving 15.83 trips per hexagon-day to 15.64 at $R^2 = 0.954$. Both sit on daily grids, correct a historical mean rather than predicting demand outright, and need no curved kernel. The operational margin is small: roughly two trips per area per day at the better of the two, and about a fifth of a trip per hexagon at the other.

4.2.2 Deep Learning Model

For the neural network, we used a feedforward network that predicts log1p-transformed trip counts, which are then back-transformed via `expm1` for evaluation. We trained on the full training split without any zero/non-zero balancing, since our experiments showed that balanced sampling actually hurt the FNN’s performance. For all six levels we used the same feature set, combining basic temporal and location features with all 14 POI categories and 5 weather variables. We did not test different feature set combinations as we did for the SVM, mostly due to computational constraints. The network uses ReLU activations in the hidden layers, a linear output unit, MSE as the loss function, and AdamW with a weight decay of $1e-4$. Training ran for up to 15 epochs with a batch size of 250. Within each training run, we used early stopping with a patience of 3 to select the best checkpoint based on validation loss, preventing the model from training past its best point. To find the best architecture per level, we ran a grid search over four layer configurations ([64,32,16], [128,64,32], [128,64,32,16], [256,128,64,32]), three learning rates (0.01, 0.001, 0.0001), and two dropout rates (0.0, 0.2), giving 24 configurations per level. The winning configuration per level was selected by validation MAE in count space. The final performance of each winning model was then evaluated once on the held-out test set, with results reported in (Figure X) across MAE, RMSE and R^2 . Looking at these results, h3_7/hourly comes out as the best performing level overall. As shown in (Table X), smaller and shallower networks worked best for the denser levels like h3_7/hourly and community, while census levels needed deeper networks and lower learning rates. Census/daily was the only level where dropout helped, which fits with it being the hardest level to train given its very high share of zero-demand rows. (Table X) shows the FNN results across all six spatial-temporal levels. The model beats the historical-mean baseline at four of the six levels, with h3_7/hourly performing best at a skill score of +6.3% MAE. The gains are small across all levels. Two levels perform notably worse: census/hourly is essentially a coin flip at -0.1%, and census/daily loses to the baseline by a large margin (-20.3% MAE, -44.7% RMSE). These are the two sparsest and most zero-inflated levels

in our dataset, and the SVM struggles there as well, which suggests the difficulty lies in the data rather than the model choice. Overall, the FNN performs best at finer temporal resolution and denser spatial units, while coarser and sparser levels remain hard to predict beyond a simple historical average. (Check after training)

4.2.2.1 Validation Results

4.2.3 Model Comparison

Both models types are compared on the same held-out test split, at H3 resolution 7 and at both temporal bins, against the same historical-mean baseline. Census and community levels are excluded: [GAP]the SVM was scored there, but no matching neural network prediction file exists, so a comparison at those levels would not be like-for-like. The comparison runs along six lines, each asking a different question about where a model's error actually falls. Supporting figures and tables are collected in Section A.

- **Aggregate accuracy.** MAE, RMSE and R^2 for the baseline, the SVM and the network at each temporal resolution, establishing whether either model beats the historical mean at all and by how much (Table 8).
- **Predicted against actual.** Predicted versus observed demand per cell and bucket, which exposes systematic over- or under-prediction and the demand range at which the fit degrades, read as spread away from the diagonal (Figure 10).
- **Spatial error.** Error aggregated per hexagon and mapped three ways: absolute error (Figure 11), error relative to each cell's own mean demand (Figure 12), and the share of each cell's day-to-day variation the model explains (Figure 13). The absolute and relative views answer different questions, since the largest absolute errors concentrate downtown purely because demand is largest there, while the relative view is what makes the quiet outer cells legible at all.
- **Head-to-head by location.** The per-hexagon difference $MAE_{NN} - MAE_{SVM}$ mapped directly, so that which model wins where is read off a single diverging scale rather than inferred by comparing two separate colour ramps (Figure 14).
- **Temporal error.** Error by day of week (Figure 15,
 - 1) and, on the hourly grid only, by hour of day (Figure 17, Figure 18), each in absolute terms and relative to mean demand. This separates error that simply tracks demand volume from error concentrated at particular times, such as the overnight hours where demand is near zero and small absolute misses dominate the relative figure. City-wide predicted and actual demand across the full test period is plotted alongside as a sanity check on aggregate level and drift (Figure 19).
- **Error by segment.** Test MAE, relative error and skill score broken down by demand band (zero, 1-10, 11-100, and above 100 trips) and by weather (dry against rainy days), as a table (Table 9) and as relative error per segment (Figure 20). This is the decisive view for fleet planning, because a model that wins on average can still lose on exactly the busiest cells, or on the rainy days, that matter operationally.

4.2.4 Shortfalls and Improvement Levers

Neither family clears the historical mean by a wide margin; the limits split into three groups: SVM-specific, network-specific, and shared. For the SVM, one failure mode is specific and fixable: the residual model underperforms the baseline on true-zero-demand cells, since the correction term does not shrink to zero where it should, and these rows dominate most levels.

A floor on the reconstructed correction, or a zero/non-zero gate before regressing magnitude, would both be worth testing. Smaller gains could come from a different sample balance, since a natural-prevalence draw performed worse, or a larger sample, though SVR’s quadratic fit cost forced the balanced subsample in the first place, trading rows directly against the breadth of the tuning search. The network’s limits are search-breadth limits, not a diagnosed failure. A single feature set was used at every level, so unlike the SVM there is no evidence whether weather or POI help or hurt; repeating the four-way comparison is the most direct lever. Architecture search was bounded the same way: layer width, learning rate, and dropout across 24 configurations, holding batch size, activation, and epoch budget fixed. No permutation-importance check was run for the network, only for the direct SVR, so whether weather and POI contribute is unknown. Both models fail on the same grids. Census/hourly and census/daily, the sparsest, most zero-inflated levels, defeat both, pointing at the data rather than the model. More generally, `base_demand` already carries most of the signal a scenario-conditioned framing allows, so both compete for a thin margin atop a strong baseline. The shared levers: weather interacted with hour and location rather than entered additively, weather resolved spatially rather than drawn from one citywide coordinate (Section 3), and POI expressed as distance to the nearest attractor rather than counts inside the cell.

4.3 Smart Charging (Reinforcement Learning)

4.3.1 Executive Summary

An automated charging controller was designed and evaluated for the client’s ride-hailing EV fleet, comparing two reinforcement learning methods: Tabular Q-Learning and a Deep Q-Network (DQN). Across five independent training runs, Tabular Q-Learning reduced the average daily charging cost by roughly 22% relative to a naive always-full-power baseline, with the probability of running out of energy remaining close to that baseline. DQN achieved comparable average performance with fewer training episodes but showed substantially higher variability across runs and is not recommended for deployment at this stage. The DQN approach may become more relevant as for example the state complexity increases greatly.

4.3.2 Problem Framing & Methodology

Each vehicle returns home at 2:00 PM and must depart at 4:00 PM. During this window, an automated controller decides every 15 minutes how fast to charge the battery, choosing from four charging speeds. Electricity prices rise later in the afternoon, while the energy required for the day’s shift is unknown until departure and varies around 30 kWh. Running out of energy mid-shift causes a missed fare and a stranded vehicle is substantially more costly than a marginally higher electricity bill. Thus, the controller must balance cost against this risk. The problem was formalized as a Markov Decision Process (MDP). At each step, the model observes battery level and remaining time, and selects one of four charging speeds (0, 7.33, 14.67, or 22 kW). Each decision incurs a small electricity cost, and a \$500 penalty applies if the battery is insufficient at departure. This penalty was set far above the maximum achievable savings from smart charging (at most a few cents per day) so that the model always prioritizes reliability. The arrival battery level is drawn from a uniform distribution (0–50% of capacity) rather than assumed as always empty, reflecting realistic driver behavior and creating a genuine cost-risk trade-off. The per-kWh price coefficient rises from ~7 to ~11 cents across the eight time steps, incentivizing earlier charging where safe. Two methods were compared: Tabular Q-Learning, a lookup table well suited to this problem’s small, enumerable state space, and DQN, which replaces the table with a neural network capable of generalizing across states. Each method was trained independently five times with different random seeds and evaluated on several thousand

simulated days per run, to avoid drawing conclusions from a single, potentially lucky training outcome.

4.3.3 Results

4.3.3.1 Tabular Q-Learning

Training over 150,000 simulated days shows the training score improving and then stabilizing, with the failure rate dropping sharply to a low, stable level (Figure 21, Figure 22), indicating convergence to a consistent policy. The learned policy (Figure 23) uses both intermediate charging speeds across a substantial share of states rather than only the extremes, indicating a genuinely differentiated strategy. Tabular Q-Learning reduced the average daily cost from \$1.92 to $\$1.50 \pm \0.02 while keeping the failure probability ($0.11\% \pm 0.09\%$) close to the naive baseline (Table 10). Overall, a roughly 22% cost reduction without a measurable increase in stranded-vehicle risk.

4.3.3.2 Deep Q-Network

DQN's training curves (Figure 24, Figure 25) oscillate throughout the full 20,000-episode period rather than stabilizing, so the best-performing weights observed at any point were retained rather than the final ones. Increasing episodes further did not help, which is also why DQN required far fewer episodes than the tabular approach (20,000 vs. 150,000), reducing the compute time needed drastically. The retained policy (Figure 26) charges at full power in almost all states, differing from the naive baseline in only two narrow state regions. Largely reproducing the naive strategy rather than a materially different one. The DQN heatmap appears fully populated while the Q-table heatmap only covers a diagonal band, because the Q-table stores entries only for visited states, whereas the neural network returns an output for any input, including states never encountered during training. DQN reduced the average daily cost to $\$1.46 \pm \0.24 with a failure probability of $1.16\% \pm 0.96\%$ which is comparable on average to Tabular Q-Learning, but with much greater variability across runs (Table 11). The per-seed variation is one of the key findings: one run reproduced the naive strategy almost exactly, while the other four achieved lower costs at markedly higher failure rates (Table 12). This inconsistency, more than the average result, is why DQN is not recommended for production use at this stage, as we are able to achieve comparable or even better results with noticeably less effort.

5 Discussion and Outlook

5.1 Summary of Findings & Implications for the Client

Across both reinforcement learning approaches, Tabular Q-Learning proved the more suitable method for the client's charging problem: it converged reliably across all training runs, reduced the average daily charging cost by roughly 22% relative to the naive baseline, and kept the probability of running out of energy close to that baseline. This directly supports the client's goal of operating a dependable ride-hailing service without compromising vehicle availability. Deep Q-Network achieved a comparable average cost and failure rate, but its substantially higher variability across runs makes it a less predictable basis for a service where a single stranded vehicle translates into a missed fare and reputational cost for the platform. Based on these results, Tabular Q-Learning is recommended for the initial fleet rollout, offering the client a solution that is both cost-effective and lightweight to maintain in production. As the client's fleet grows or charging decisions are greatly extended, DQN may become more relevant, positioning it as a natural next step for later stages of the platform's expansion rather than for the current use case.

5.2 Limitations & Future Work

These results should be read as directional, not final. They come from Chicago’s existing taxi trade rather than the client’s own electric fleet, and the strongest demand models here only modestly beat a simple historical average, so cell and calendar information already explain most of what this data can tell us. The charging recommendation likewise rests on a fixed daily price curve and a single vehicle, rather than a full fleet drawing on a real, time-varying market. Validating both the demand patterns and the charging economics against the client’s own operating data, once it exists, is the one step that would do more than any further modelling to confirm these findings hold in practice.

A Additional Figures and Tables

This appendix collects supporting figures and tables that are not reproduced in the main body of the report, organized by the notebook that produces them (see the pipeline overview in the project `CLAUDE.md`).

A.1 Taxi Data Preparation (1.1)

No additional labeled figures or tables from this notebook.

A.2 Weather Data Preparation (1.2)

Table 1: Selected Meteorological Variables

Variable	Description	Datatype
2m Temperature (K and °C)	Air temperature at 2 meters above the surface. Expected to drive customer demand (e.g., fewer walk/bike substitutes in extreme cold) and EV battery performance, since range may drop at low temperatures.	float64
Total Precipitation (m and mm)	Accumulated liquid and solid precipitation reaching the surface. Expected to increase short-distance ride demand and reduce walking/cycling alternatives.	float64
10m Wind Components u10 and v10	Eastward and northward horizontal wind components at 10 meters above ground level. Combined into wind speed and direction (see Feature Engineering); relevant for extreme-weather demand spikes and outdoor-wait discomfort.	float64
Snow Cover / Snow Depth	Fraction of the grid cell covered by snow, and mean snow thickness on the ground (excluding canopy snow). Expected to directly affect road conditions, trip duration, and safety-related demand drops, and are key drivers of charging infrastructure planning in winter months.	float64

No additional labeled figures or tables from this notebook.

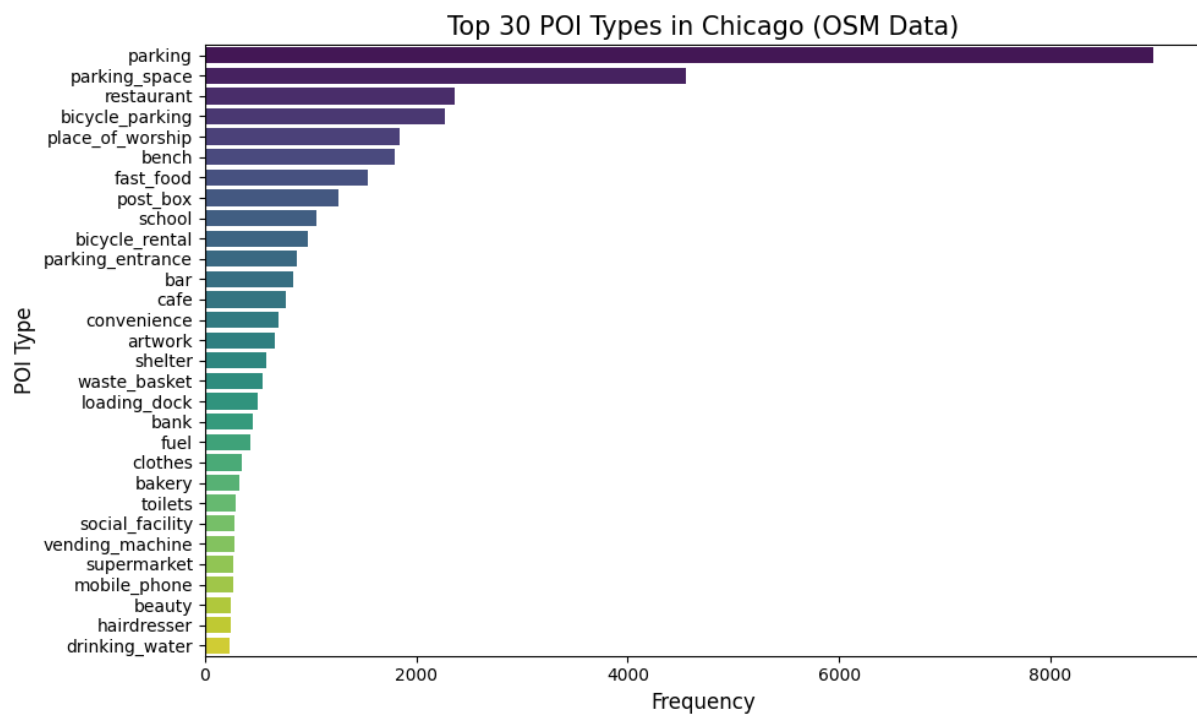


Figure 1: Top 30 most frequent raw OSM POI types in Chicago, before category mapping.

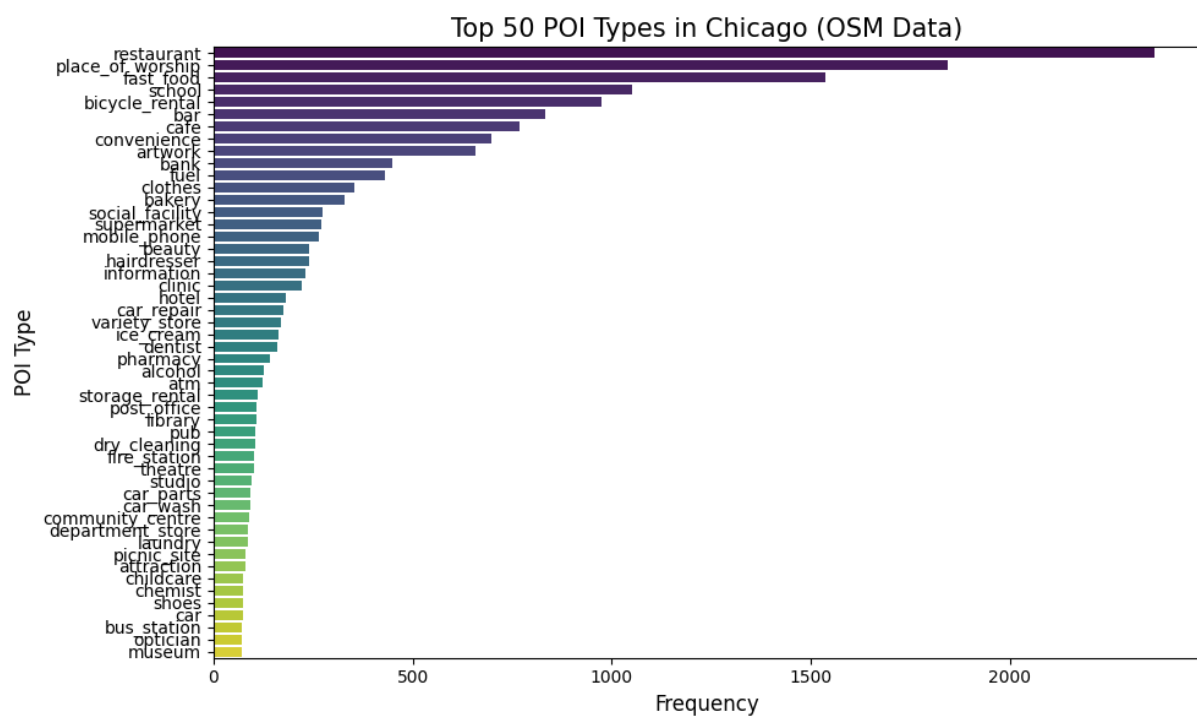


Figure 2: Top 50 most frequent POI types remaining after category filtering (unmapped 'other' types dropped).

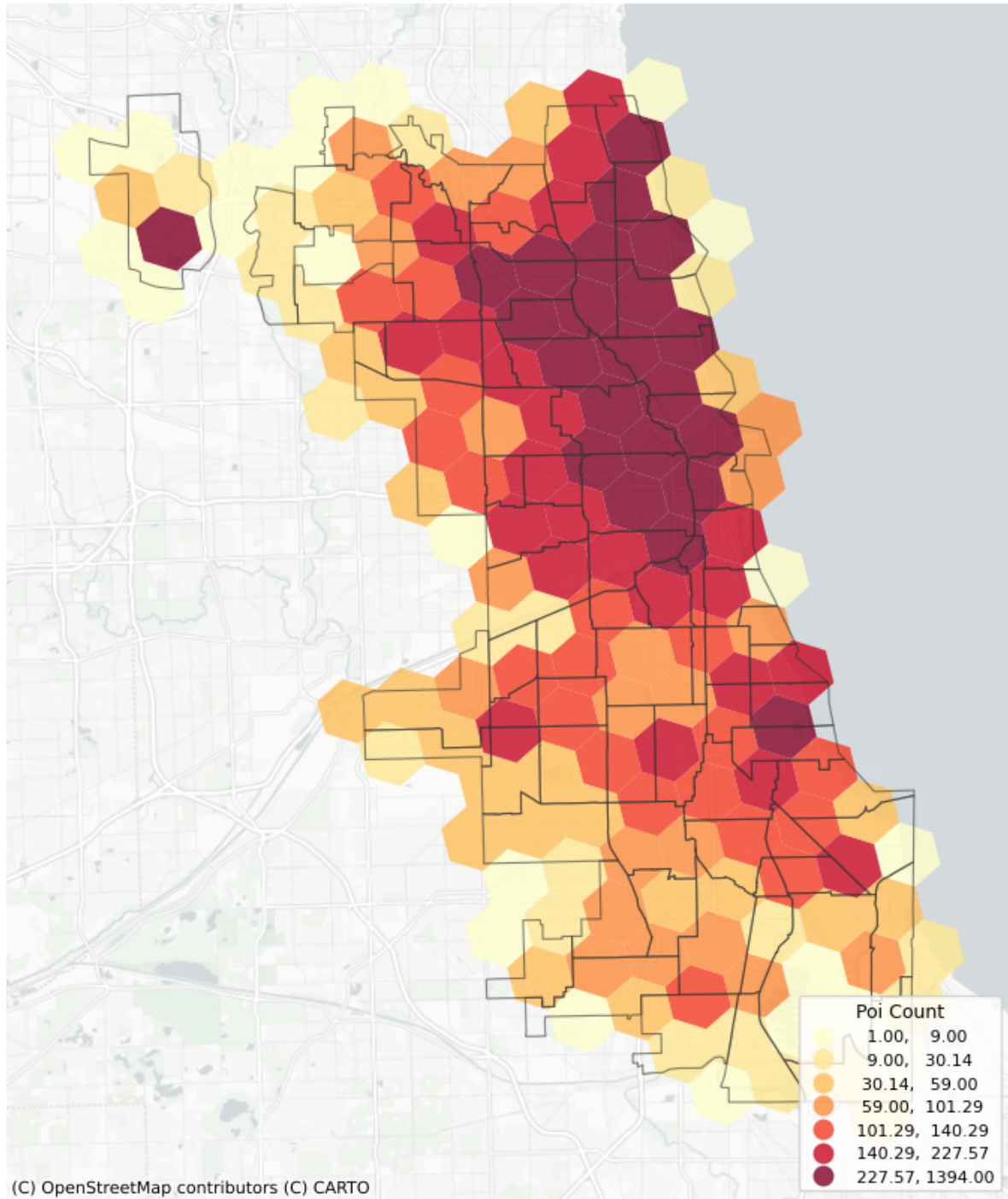
Chicago — POI Count by H3 Hexagon (resolution 7)

Figure 3: Total POI count per H3 hexagon (resolution 7).

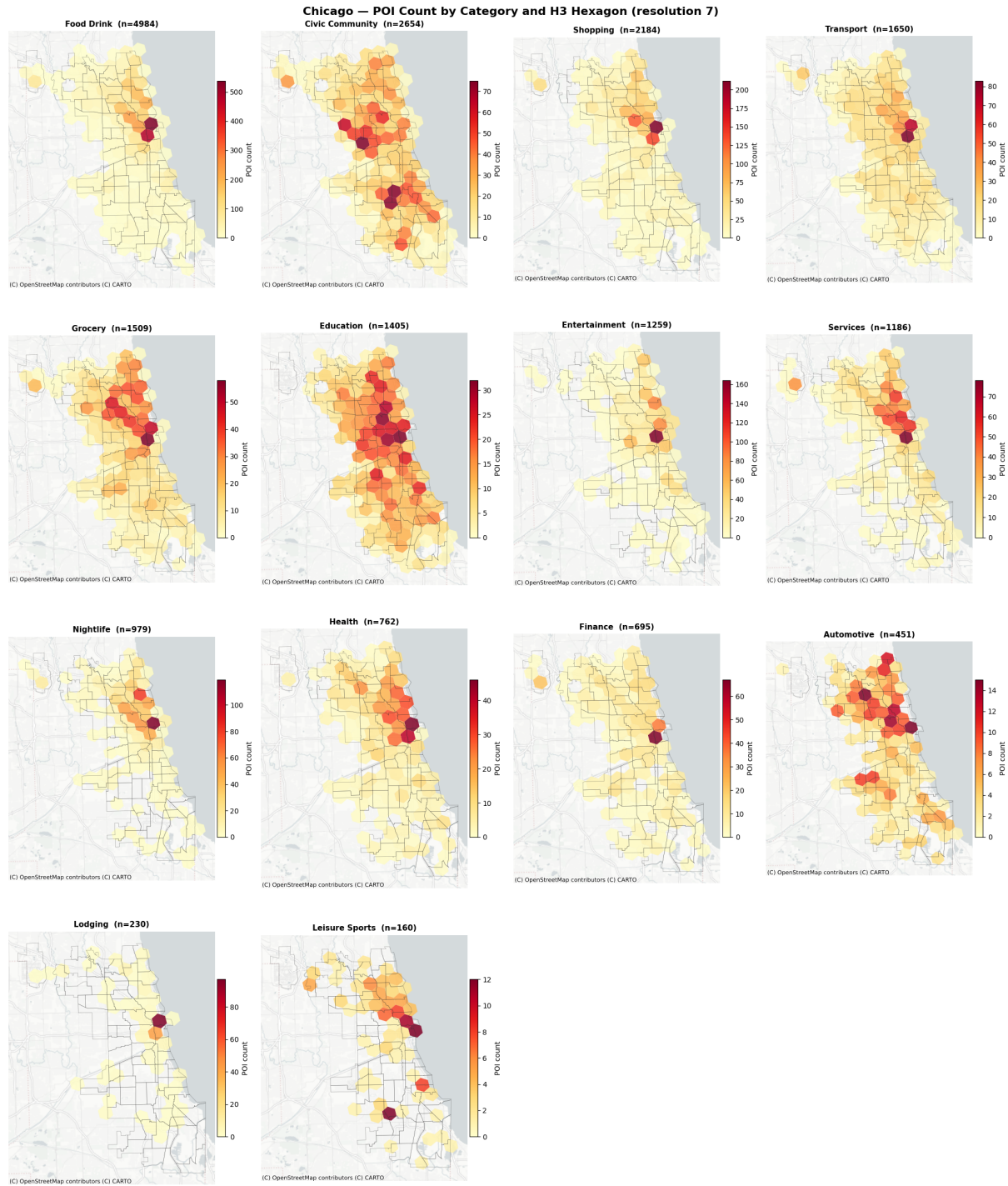


Figure 4: POI count per H3 hexagon (resolution 7), faceted by category.

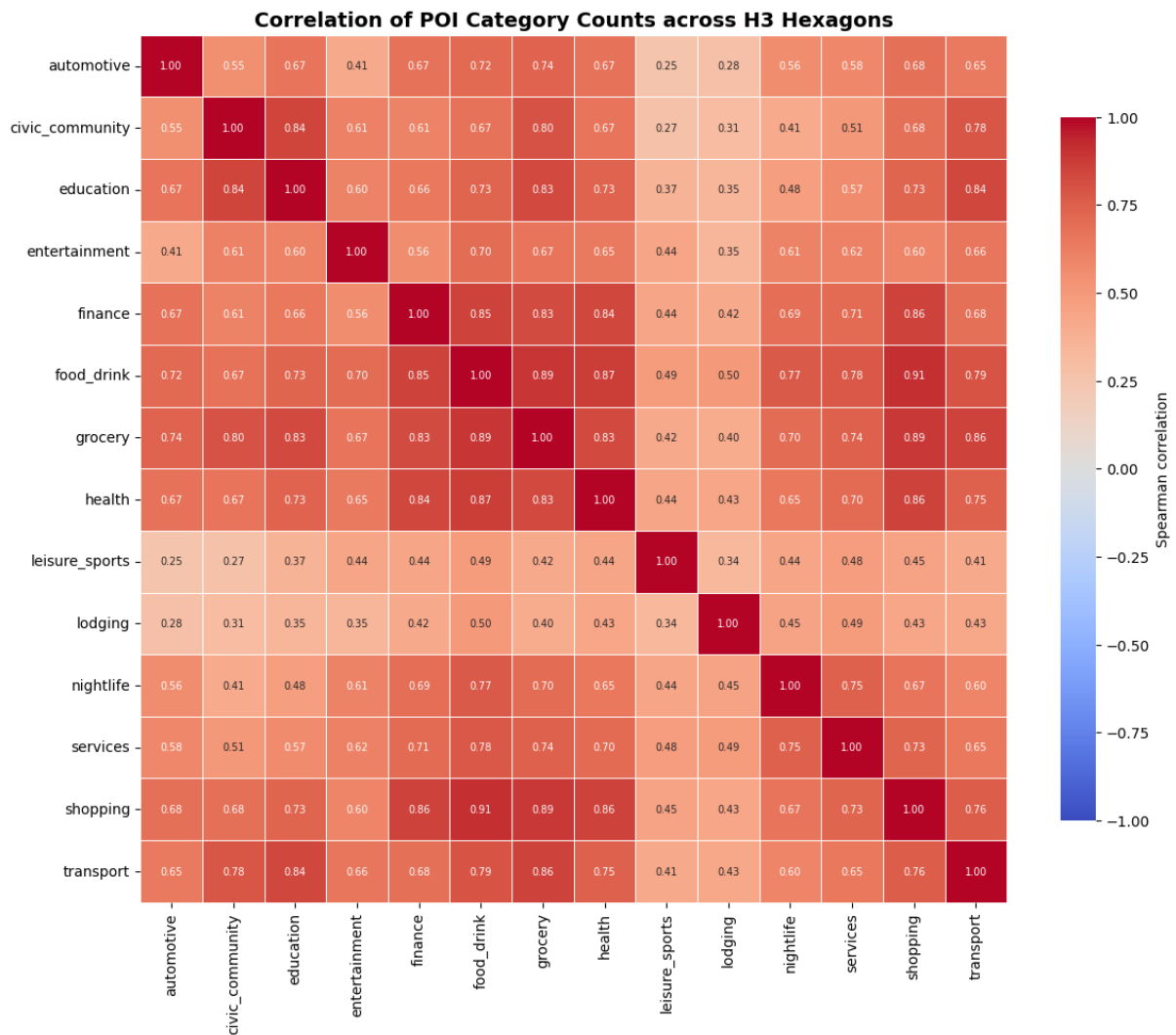


Figure 5: Spearman correlation of POI category counts across H3 hexagons (resolution 7).

A.3 Points of Interest Data Preparation (1.3)**A.4 Temporal-Spatial Discretization (1.4)**

No additional labeled figures or tables from this notebook.

A.5 Descriptive Analysis: Spatial (2.1)

No additional labeled figures or tables from this notebook.

A.6 Descriptive Analysis: Temporal (2.2)

No additional labeled figures or tables from this notebook.

A.7 Descriptive Analytics: Weather (2.4)

No additional labeled figures or tables from this notebook.

A.8 Feature Engineering (3.1)

Table 2: List of all Engineered Features.

Engineered Feature	Description	DataType
distance_to_loop	Great-circle (haversine) distance from each grid cell's coordinates to the Chicago Loop (downtown reference point), used as a proxy for centrality/urban density.	float64
wind_speed	Magnitude of the horizontal wind vector, computed as $\sqrt{u_{10}^2 + v_{10}^2}$ from the 10m u/v wind components.	float64
wind_dir	Meteorological wind direction (direction the wind blows from), computed via $\text{atan2}(u_{10}, v_{10})$ and converted to compass degrees (0-360°).	float64
is_holiday	Binary indicator flagging official U.S. federal holidays observed in Illinois, expected to shift ride-hailing demand relative to regular weekdays.	int64
is_near_holiday	Binary indicator flagging the calendar day immediately before or after a public holiday, capturing anticipatory or spillover demand effects.	int64
hour_of_day	Hour extracted from the timestamp, capturing intraday demand cycles. Only present in hourly-resolution grids.	int64
hour_sin	Sine transformation of hour_of_day, encoding time of day on a continuous circular scale. Only present in hourly-resolution grids.	float64
hour_cos	Cosine transformation of hour_of_day, paired with hour_sin to avoid the discontinuity between hour 23 and hour 0. Only present in hourly-resolution grids.	float64
month	Calendar month extracted from the timestamp, capturing seasonal variation.	int64
month_sin	Sine transformation of month, encoding the annual cycle continuously.	float64

Table 2: List of all Engineered Features.

Engineered Feature	Description	DataType
month_cos	Cosine transformation of month, paired with month_sin to avoid the discontinuity between December and January.	float64
day_of_week	Numeric day of week extracted from the timestamp (0 = Monday, ..., 6 = Sunday), capturing weekly demand patterns.	int64
is_weekend	Binary indicator flagging Saturday and Sunday (day_of_week $\geq$ 5), expected to correlate with different ride purposes (leisure vs. commute).	int64
weekday	Numeric day of week extracted from the timestamp; identical in definition to day_of_week (0 = Monday, ..., 6 = Sunday).	int64
season	Meteorological season derived from month, encoded as an integer (1 = Winter, 2 = Spring, 3 = Summer, 4 = Autumn), capturing broader seasonal demand and weather regimes.	int64
season_sin	Sine transformation of season, representing the seasonal cycle continuously.	float64
season_cos	Cosine transformation of season, paired with season_sin to avoid a discontinuity between the last and first season of the year.	float64
base_demand	Historical average of Total_Trip_Start, computed on the training split only and grouped by spatial cell, month, day_of_week, and (for hourly grids) hour_of_day; unseen key combinations are filled with the global training mean.	float64

A.9 SVR Hyperparameter Search Grid (3.2)

Both SVR notebooks share the identical two-stage search defined in 3.2 `PredictionCommonGround.ipynb`, so the direct and residual results stay directly comparable. The grid search runs only for the kernel that wins the scan, not for all three.

Table 3: Two-stage SVR hyperparameter search grid, shared by the direct (3.2a) and residual (3.2b) models.

Stage	Setting	Values searched
Kernel scan	Kernels	<code>linear</code> , <code>poly</code> , <code>rbf</code>
	Held fixed during the scan	<code>C = 1</code> , <code>gamma = "scale"</code>
	Selection criterion	Validation R^2 in count space
Grid search	Training sample	4,000 rows per class (8,000 total)
	<code>C</code>	1, 10, 50
	<code>gamma</code>	"scale", 0.05, 0.2
	<code>degree</code> (polynomial kernel only)	2, 3
	Cross-validation	3-fold
	Selection criterion	Count-space MAE
	Training sample	8,000 rows per class (16,000 total)

Stage	Setting	Values searched
Feature sets	Candidates	basic , basic+poi , basic+weather , basic+poi+weather
	Selection criterion	Validation R^2 in count space
Scoring	Validation subsample	50,000 rows; the level's winner is re-scored on the full validation split

The four candidate feature sets are built from the same blocks at every level. **basic** is hour (hourly grids only) plus calendar, location and history; the other three add the POI and weather blocks to it.

Table 4: Feature blocks composing the four candidate sets. Resulting widths are 10 / 24 / 15 / 29 features on daily grids and 12 / 26 / 17 / 31 on hourly grids, for **basic** / **basic+poi** / **basic+weather** / **basic+poi+weather** respectively.

Block	Features	Count
Hour (hourly grids only)	hour_sin , hour_cos	2
Calendar	month_sin , month_cos , is_weekend , is_holiday , is_near_holiday , day_of_week	6
Location	lat , lon , distance_to_loop	3
History	base_demand	1
POI	14 poi_cat_* category counts	14
Weather	2m_temp_c , total_precip_mm , snow_cov , snow_depth , wind_speed	5

A.10 SVR: Direct Model (3.2a)

Table 5: Winning SVR per level, with skill against the historical-mean baseline.

	spatial	temporal	feature_set	kernel	C	gamma	val_MAE	val_RMSE	val_R2	base_MAE
0	h3_7	daily	basic+poi	rbf	50	scale	15.7997	88.8310	0.9494	15.8295
1	community	daily	basic+poi	rbf	10	scale	37.4429	139.8964	0.9578	36.1879
2	census	daily	basic+poi	rbf	10	scale	4.0758	27.3988	0.9341	3.8284
3	community	hourly	basic+poi	rbf	1	scale	2.7382	10.3958	0.9133	2.5250
4	h3_7	hourly	basic	rbf	10	0.05	0.9858	6.2626	0.9148	0.8861
5	census	hourly	basic+poi+weather	rbf	1	scale	0.3302	2.0741	0.8795	0.2438

A.11 SVR: Residual-on-Baseline Model (3.2b)

Table 6: Winning residual-on-baseline SVR per level, with skill against the historical-mean baseline.

	spatial	temporal	feature_set	kernel	C	gamma	val_MAE	val_RMSE	val_R2	base_MAE
0	community	daily	basic+weather	linear	1	scale	34.2541	118.3257	0.9698	36.1879
1	h3_7	daily	basic+poi	linear	1	scale	15.6367	84.9372	0.9538	15.8295

Table 6: Winning residual-on-baseline SVR per level, with skill against the historical baseline.

	spatial	temporal	feature_set	kernel	C	gamma	val_MAE	val_RMSE	val_R2	base_MAE
2	census	daily	basic+poi+weather	rbf	1	scale	3.8174	26.1691	0.9399	3.8284
3	h3_7	hourly	basic+weather	linear	1	scale	0.8838	5.5285	0.9336	0.8861
4	community	hourly	basic+poi+weather	rbf	1	scale	2.5498	9.2293	0.9316	2.5250
5	census	hourly	basic	linear	1	scale	0.3221	1.9674	0.8915	0.2438

Table 7: Direct SVR (3.2a) vs. residual-on-baseline SVR trained from scratch (this notebook), per level, on the validation split.

	spatial	temporal	feature_set	kernel	skill_MAE_direct	skill_MAE_residual	delta_skill
0	h3_7	hourly	basic+weather	linear	-0.1125	0.0026	0.1151
1	community	daily	basic+weather	linear	-0.0347	0.0534	0.0881
2	community	hourly	basic+poi+weather	rbf	-0.0844	-0.0098	0.0746
3	census	daily	basic+poi+weather	rbf	-0.0646	0.0029	0.0675
4	census	hourly	basic	linear	-0.3547	-0.3214	0.0333
5	h3_7	daily	basic+poi	linear	0.0019	0.0122	0.0103

A.12 Deep Learning Prediction (3.3)

No additional labeled figures or tables from this notebook.

A.13 SVM vs. Neural Network Performance Comparison (3.4)

H3_7/daily

H3_7/hourly

--- h3_7/daily ---

--- h3_7/hourly ---

A.14 Reinforcement Learning (4.1)

--- Calculating Naive Strategy Baseline (2000 days) ---

Naive Success Rate: 100.00%

Average Naive Charging Cost per day: \$1.92

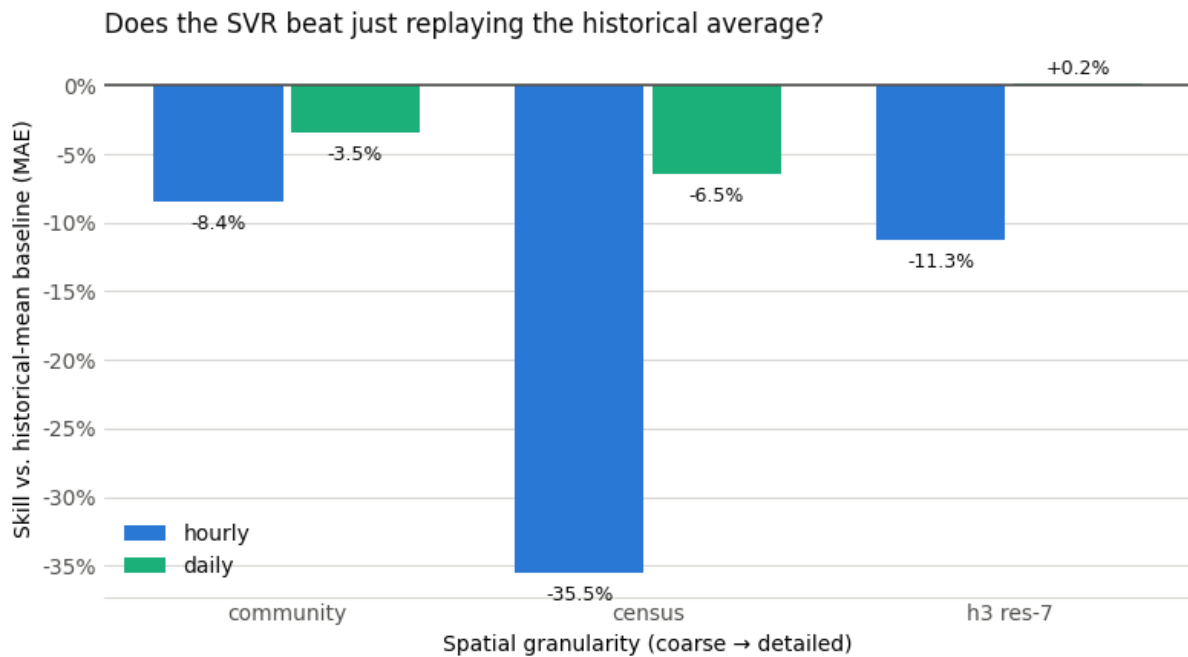


Figure 6: SVR skill against the historical-mean baseline, by spatial and temporal granularity.

Feature importance: reliance vs. necessity (= block not in this model's feature set)

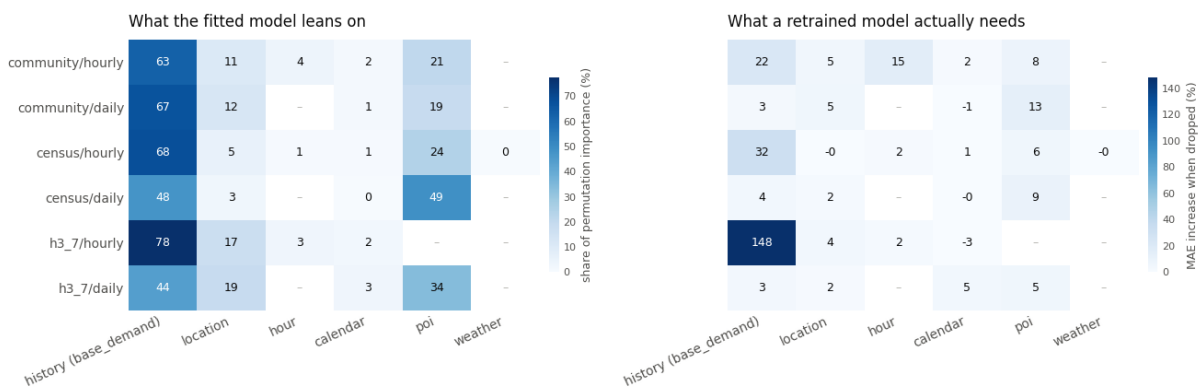


Figure 7: Left: what the fitted model leans on (grouped permutation importance, as each block's share of the level's total). Right: what a retrained model actually needs (drop-column ablation, % increase in MAE when the block is removed). The two disagree sharply on base_demand — the fitted model depends on it almost entirely, yet a model refitted without it loses very little, because lat/lon/hour/day-of-week can reconstruct it.

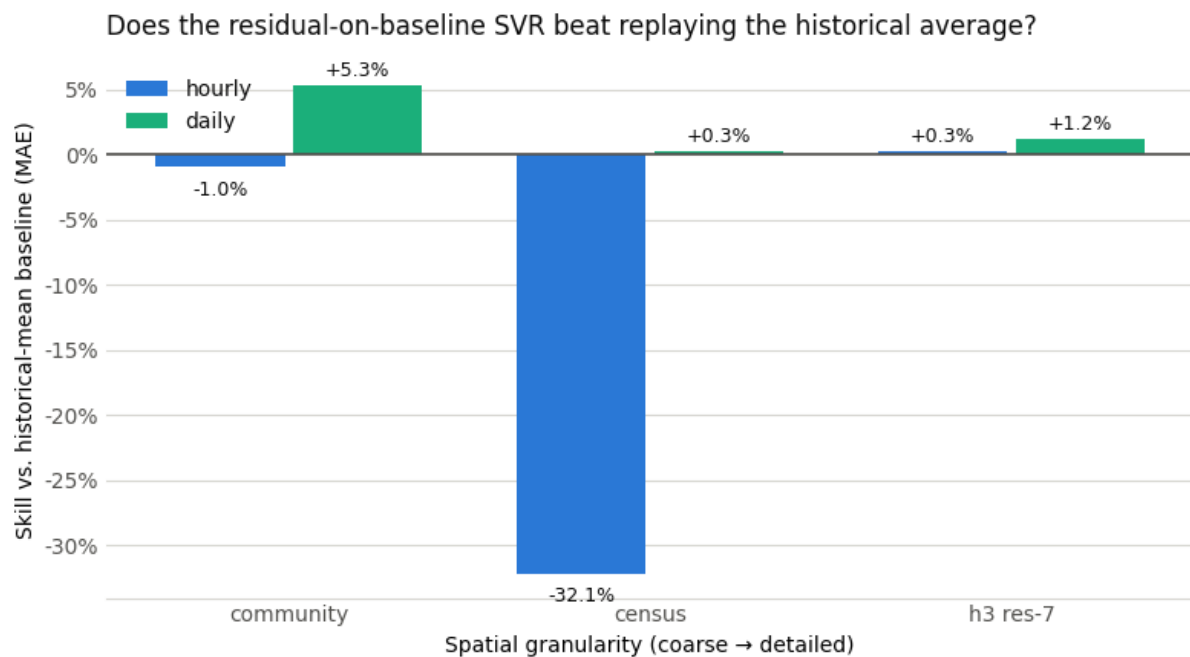


Figure 8: Residual-on-baseline SVR skill against the historical-mean baseline, by spatial and temporal granularity.

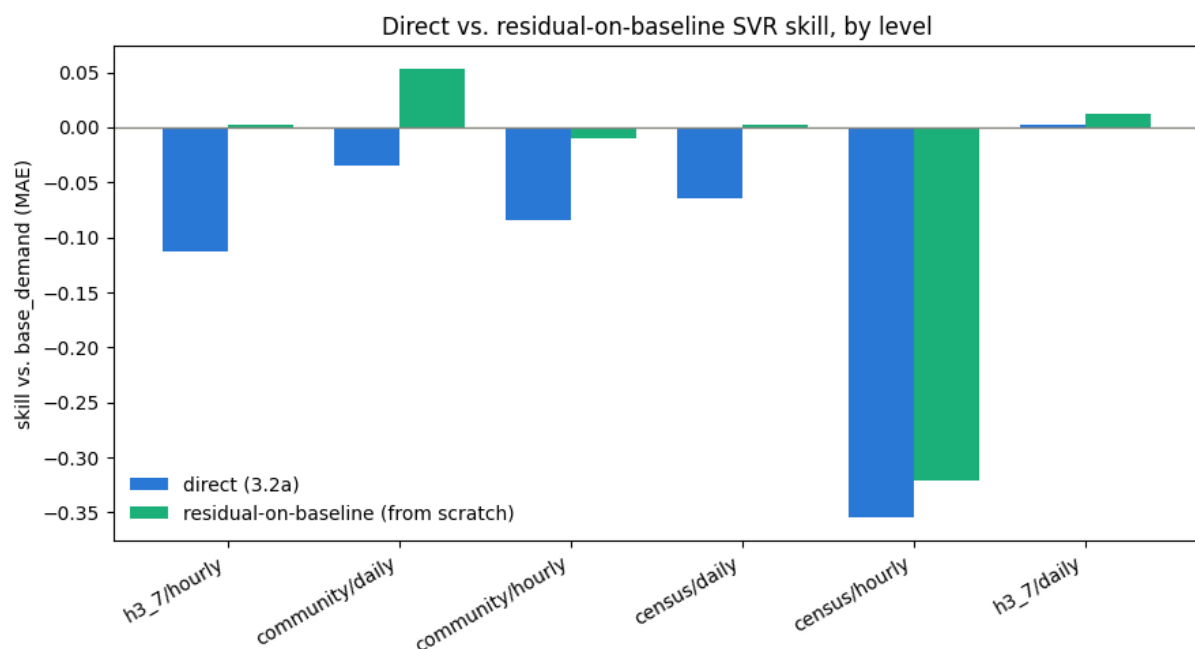


Figure 9: Direct (3.2a) vs. residual-on-baseline (this notebook) SVR skill, both independently tuned, per level.

Table 8: Overall TEST performance at h3_7/daily and h3_7/hourly.

(a) h3_7/daily			
	MAE	RMSE	R2
Model			
Baseline	21.3913	121.0296	0.9061
Best SVM (Residual SVR)	20.6391	112.3737	0.9190
Neural Network (FNN)	20.1520	118.8802	0.9094

(b) h3_7/hourly			
	MAE	RMSE	R2
Model			
Baseline	1.0678	7.4296	0.8805
Best SVM (Residual SVR)	1.0165	6.4802	0.9091
Neural Network (FNN)	0.9767	6.4700	0.9094

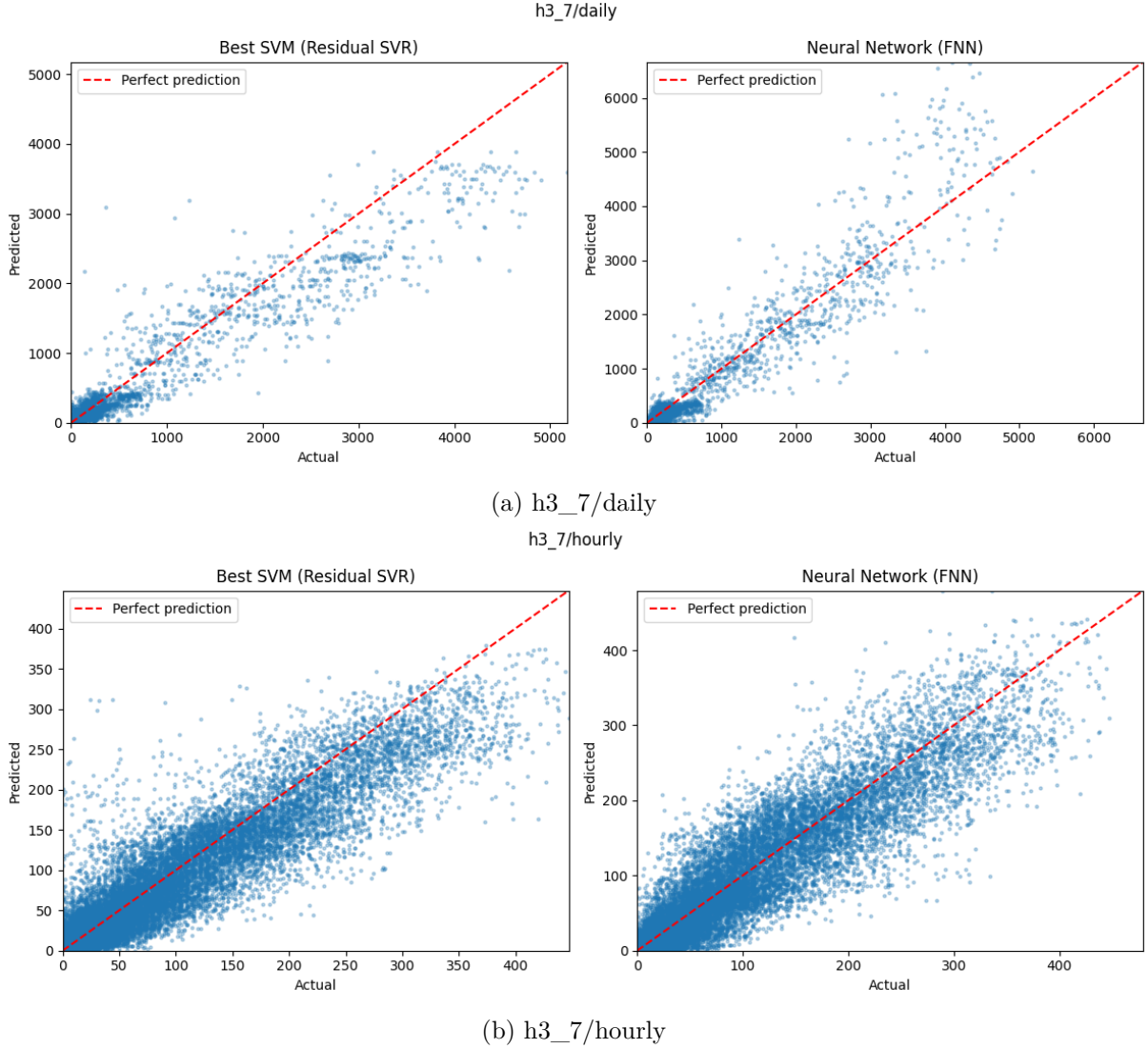
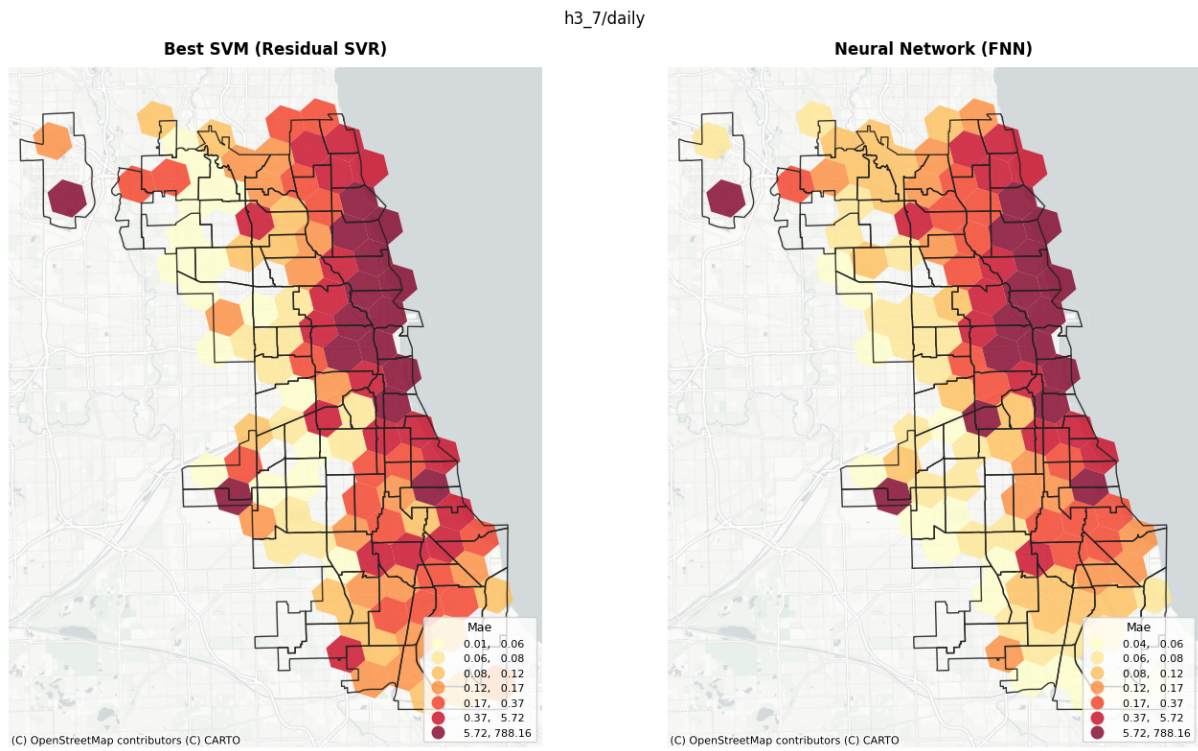
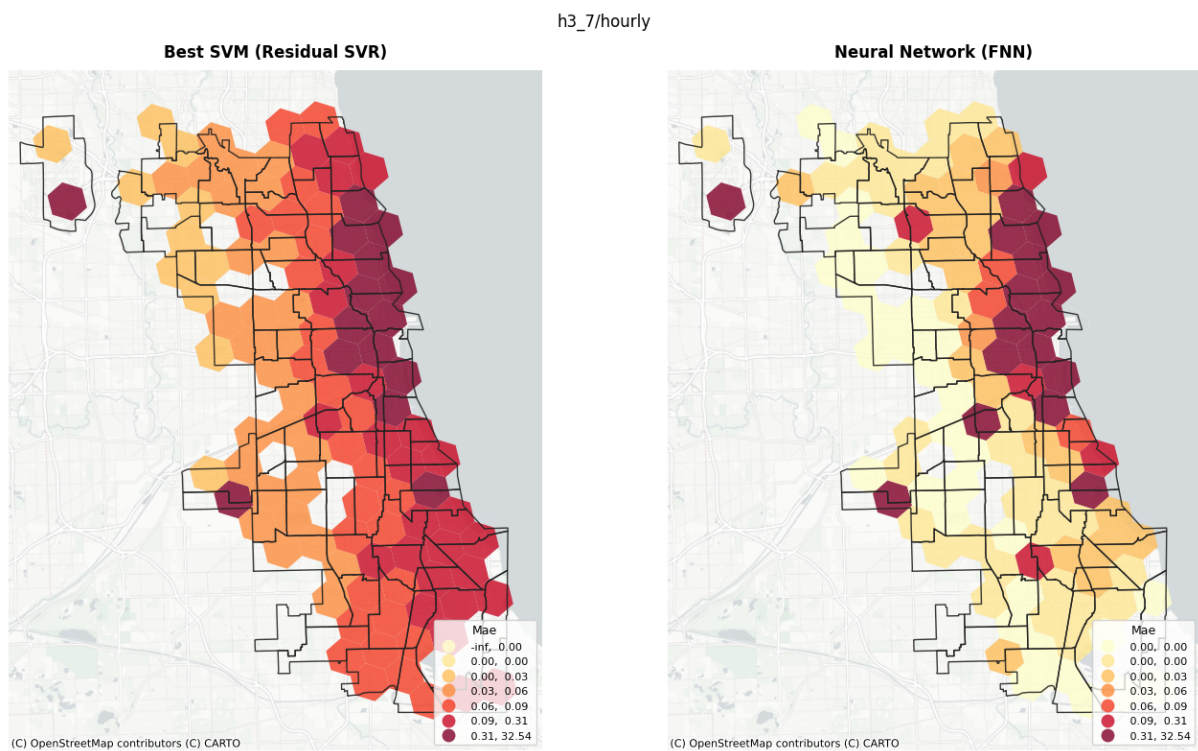


Figure 10: Actual vs. predicted demand per temporal resolution.

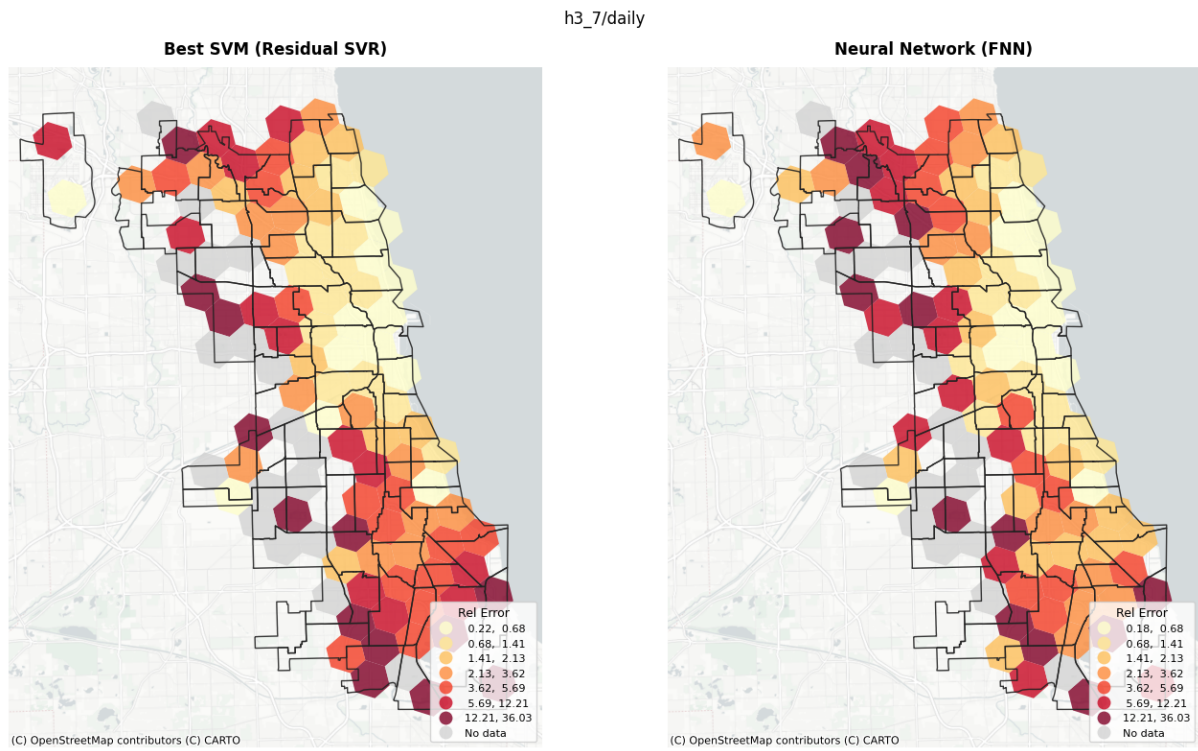


(a) h3_7/daily

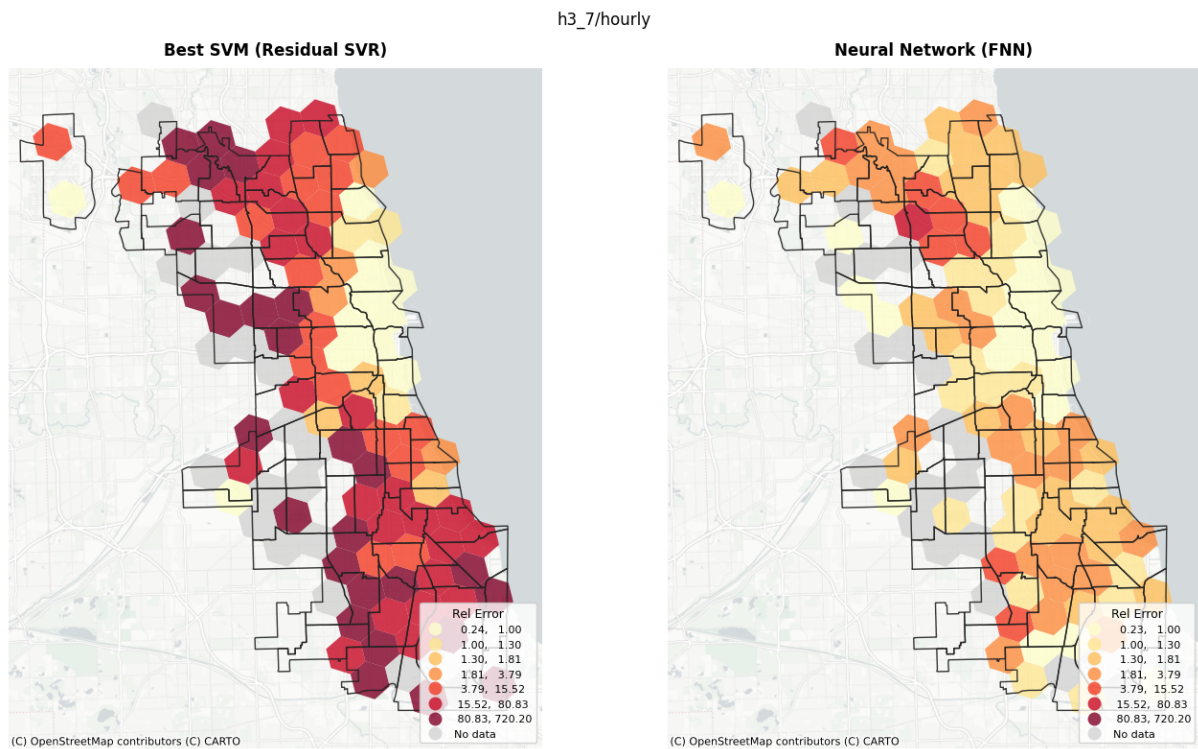


(b) h3_7/hourly

Figure 11: MAE per hexagon per temporal resolution.

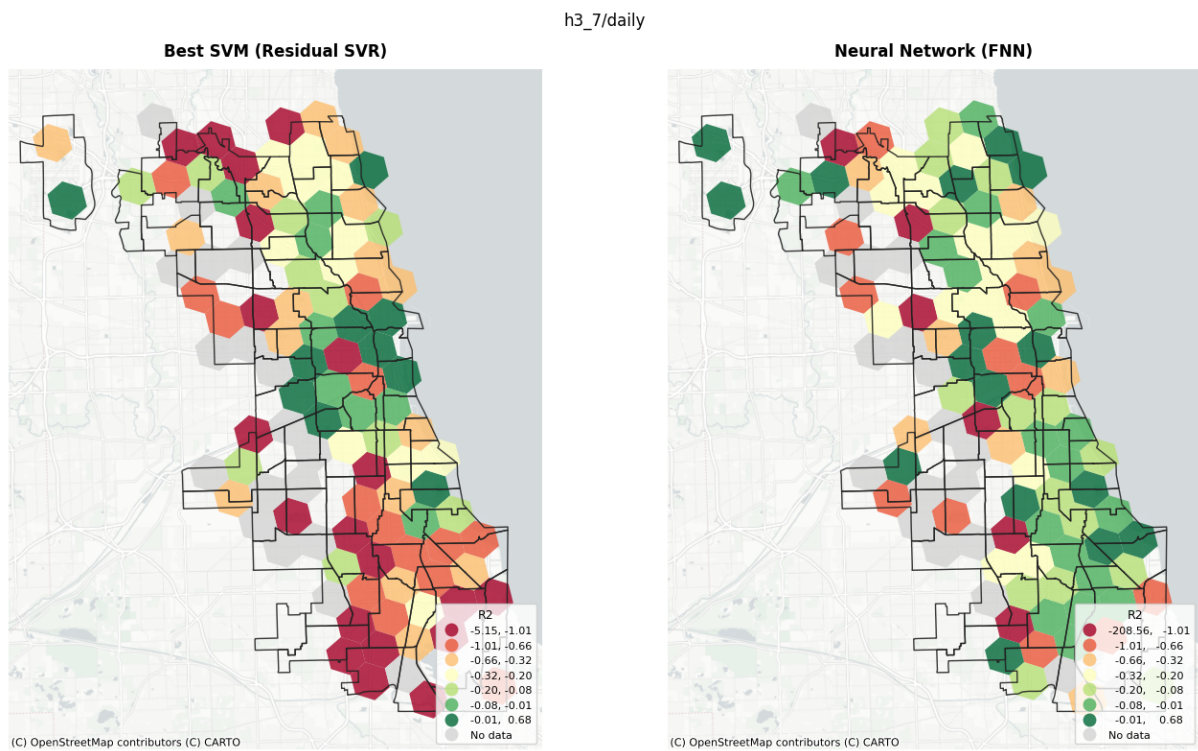


(a) h3_7/daily

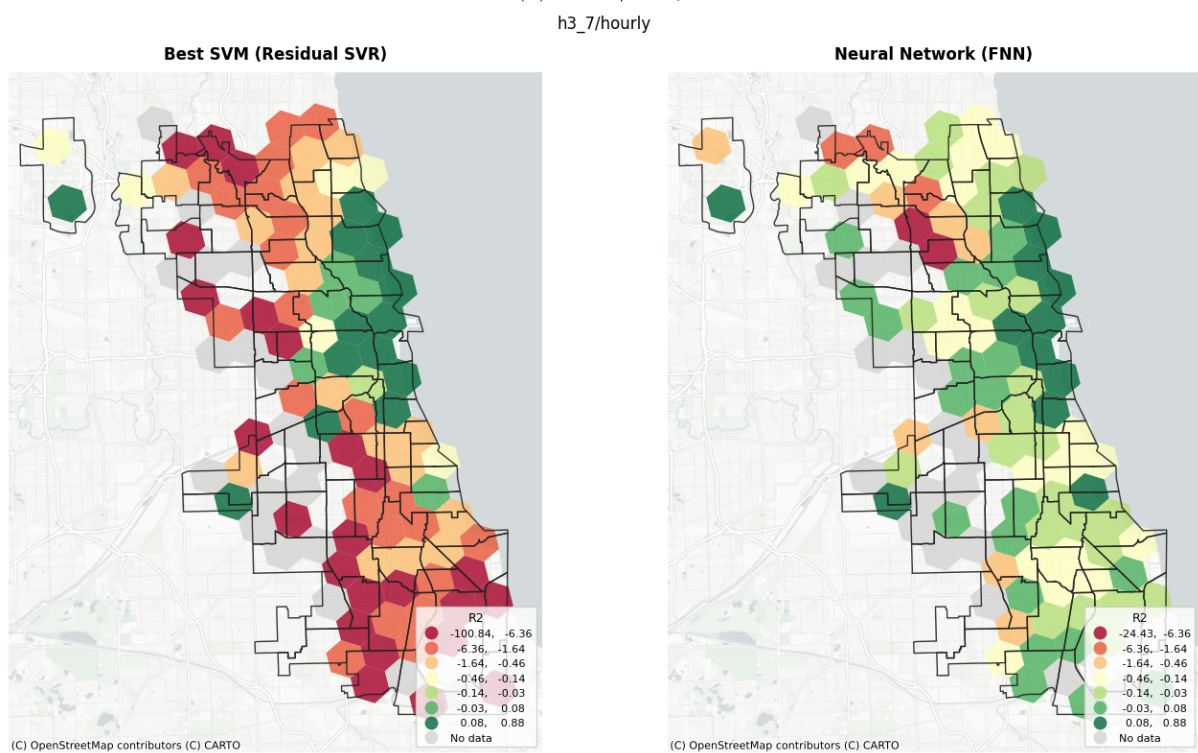


(b) h3_7/hourly

Figure 12: Relative error per hexagon per temporal resolution.

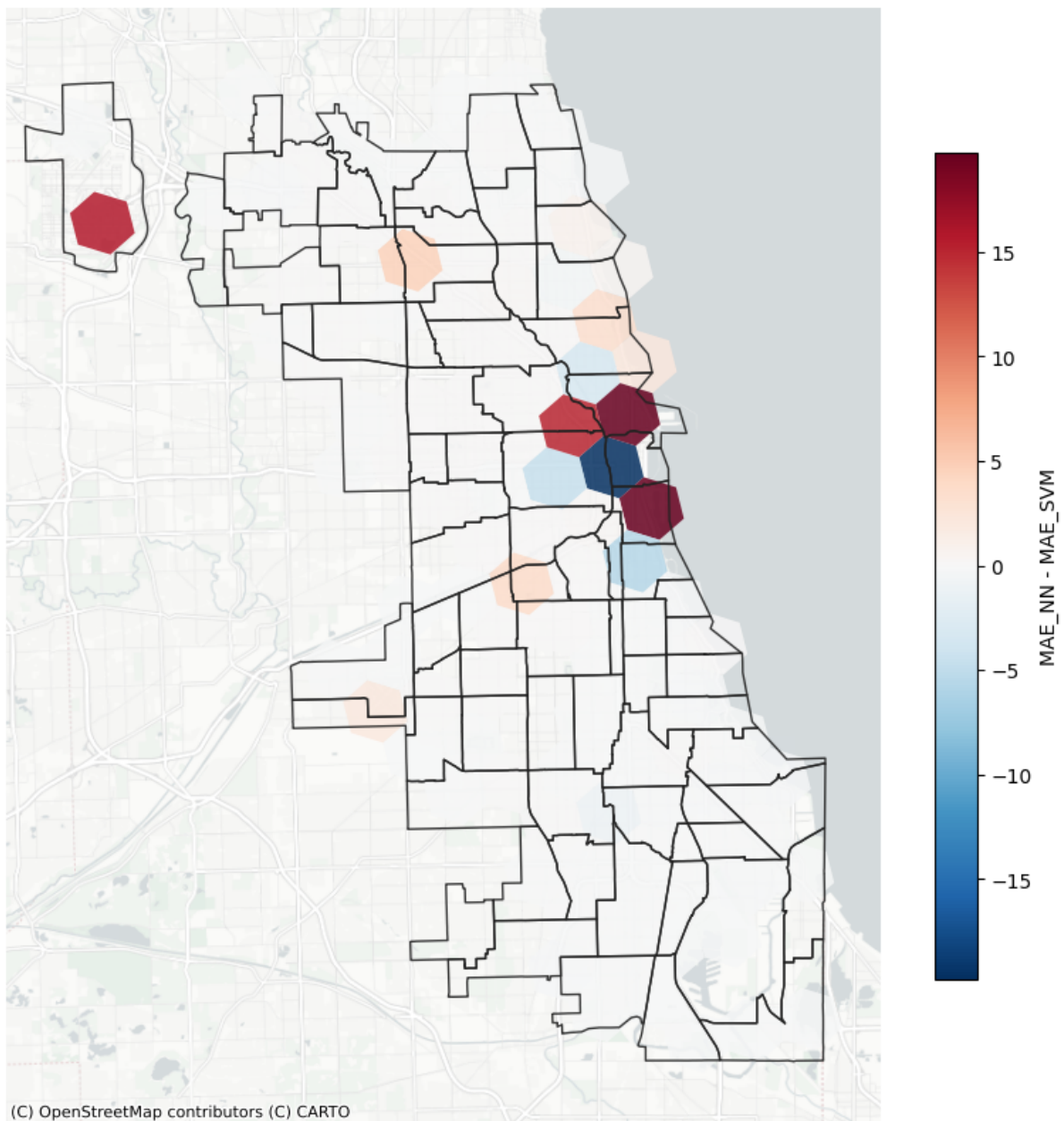


(a) h3_7/daily

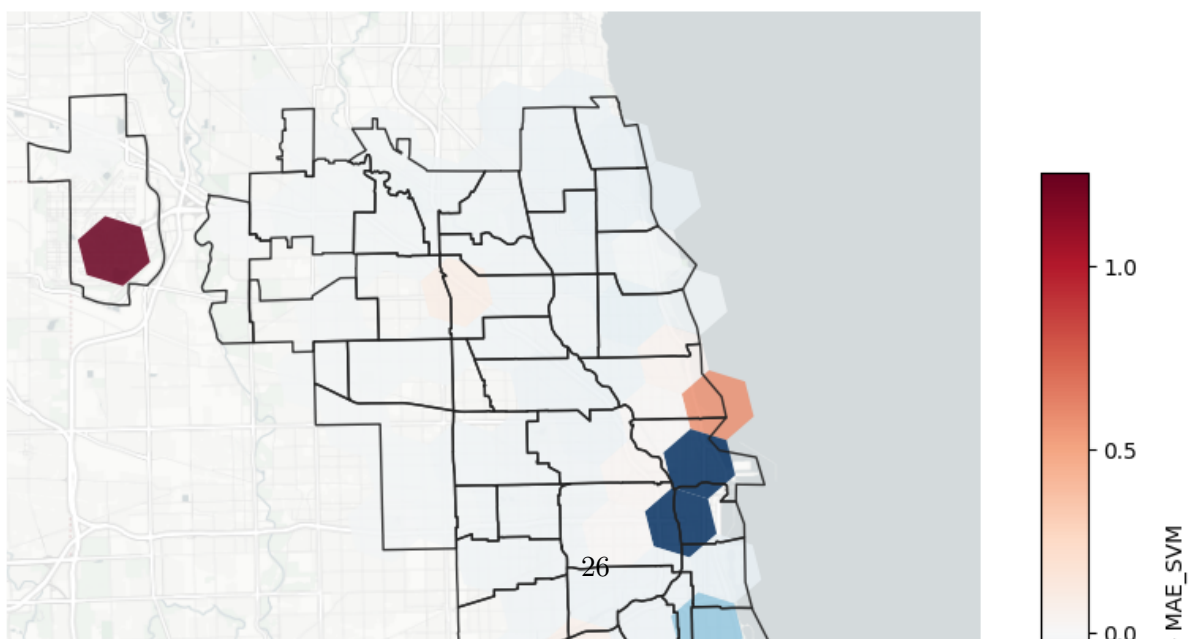


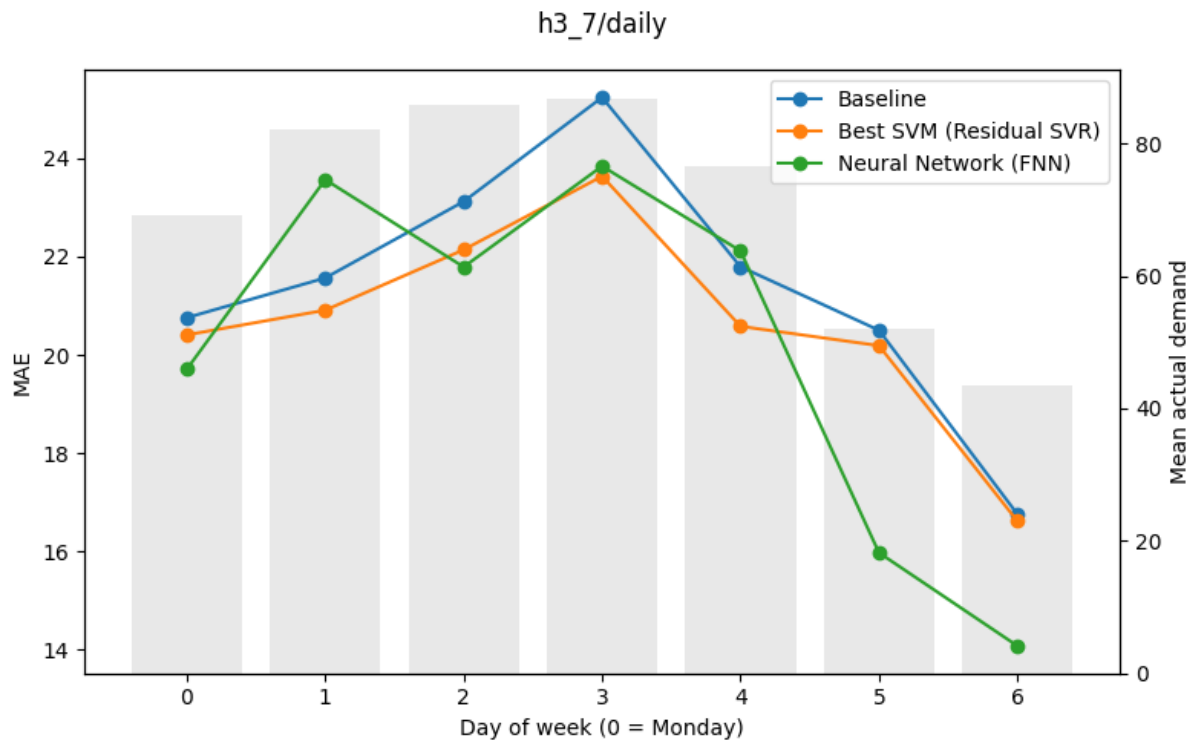
(b) h3_7/hourly

Figure 13: R^2 per hexagon per temporal resolution.

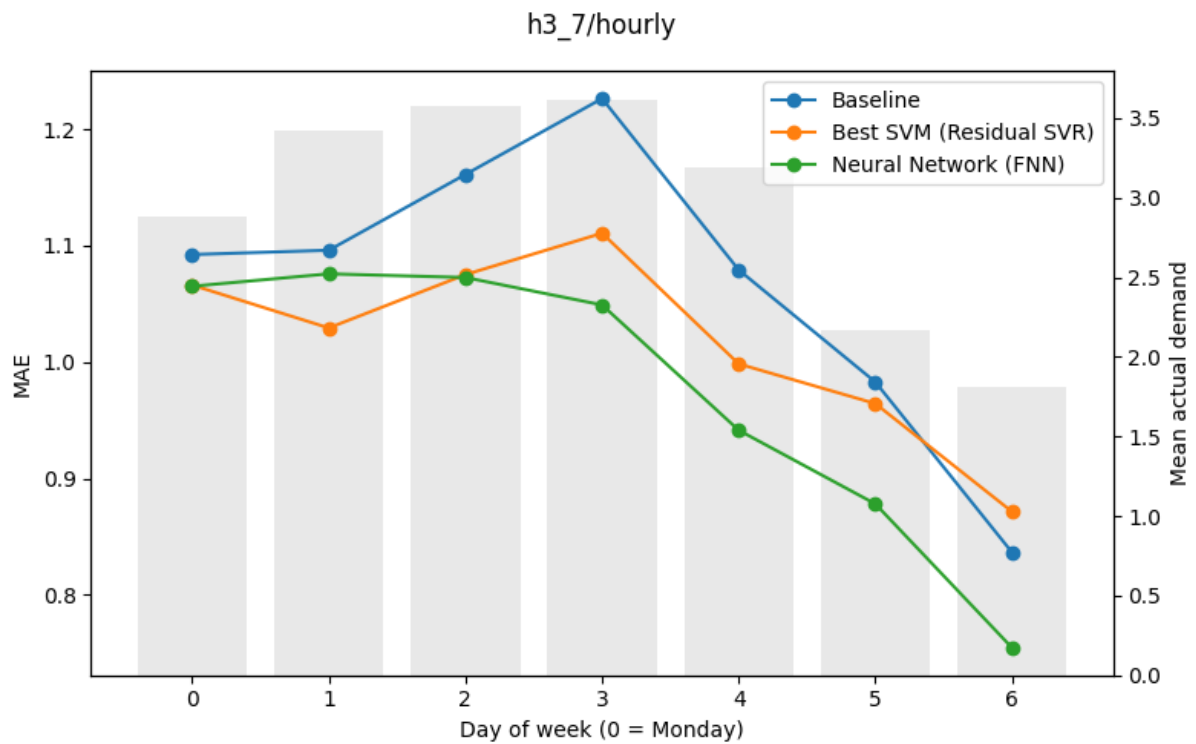
h3_7/daily — Where does SVM beat FNN?

(a) h3_7/daily

h3_7/hourly — Where does SVM beat FNN?

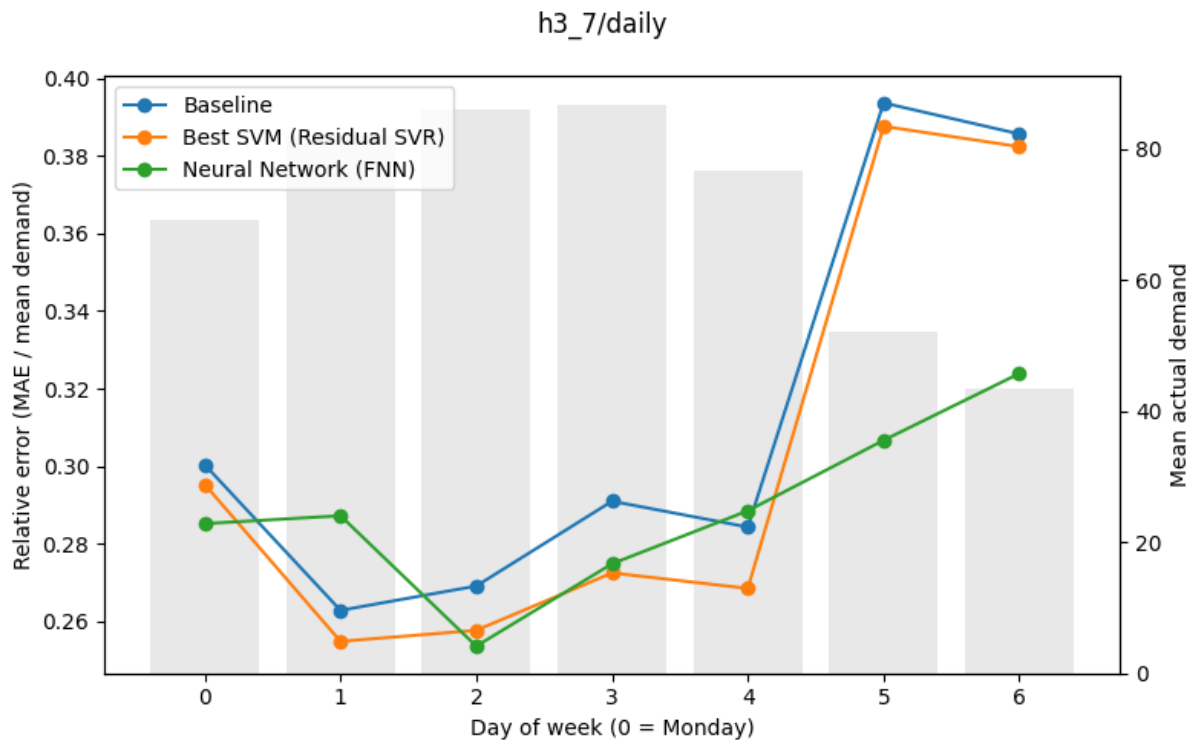


(a) h3_7/daily

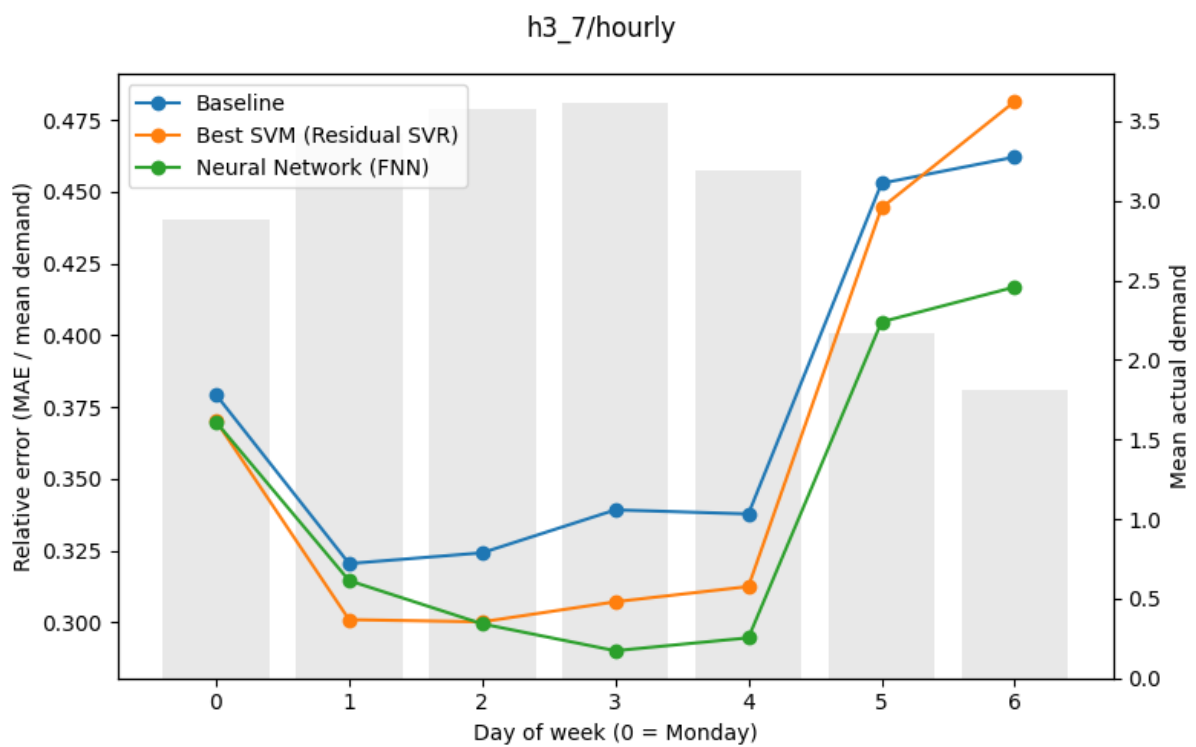


(b) h3_7/hourly

Figure 15: Test MAE by day of week per temporal resolution.



(a) h3_7/daily



(b) h3_7/hourly

Figure 16: Test relative error (MAE / mean demand) by day of week per temporal resolution.

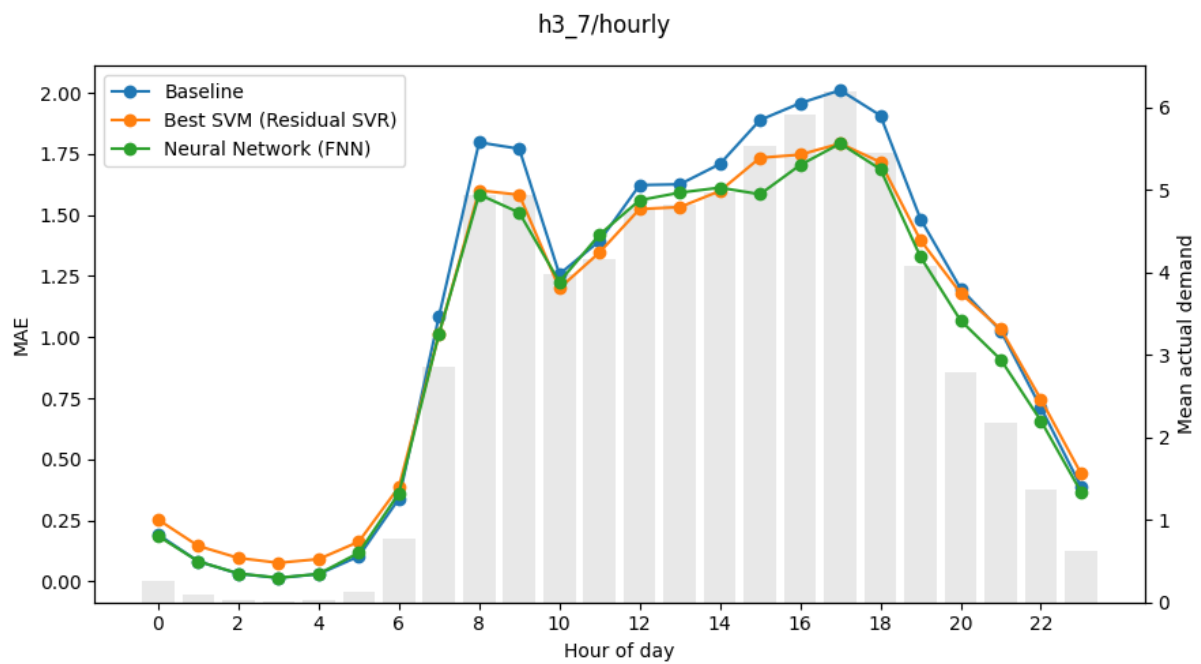


Figure 17: Test MAE by hour of day (h3_7/hourly).

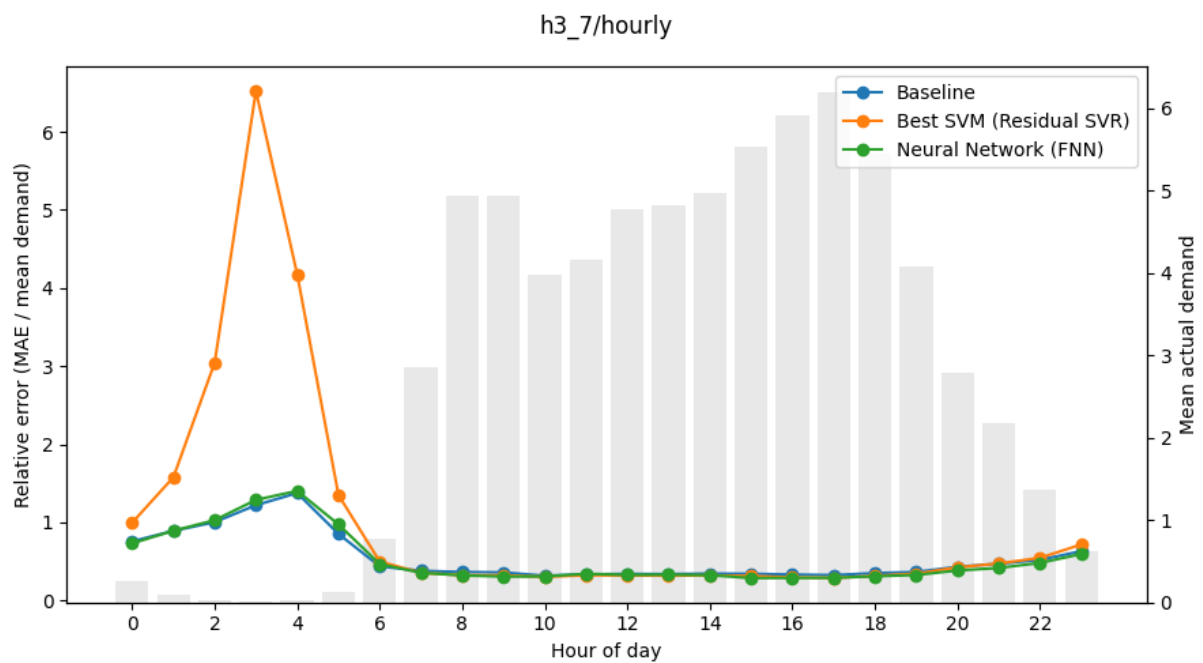
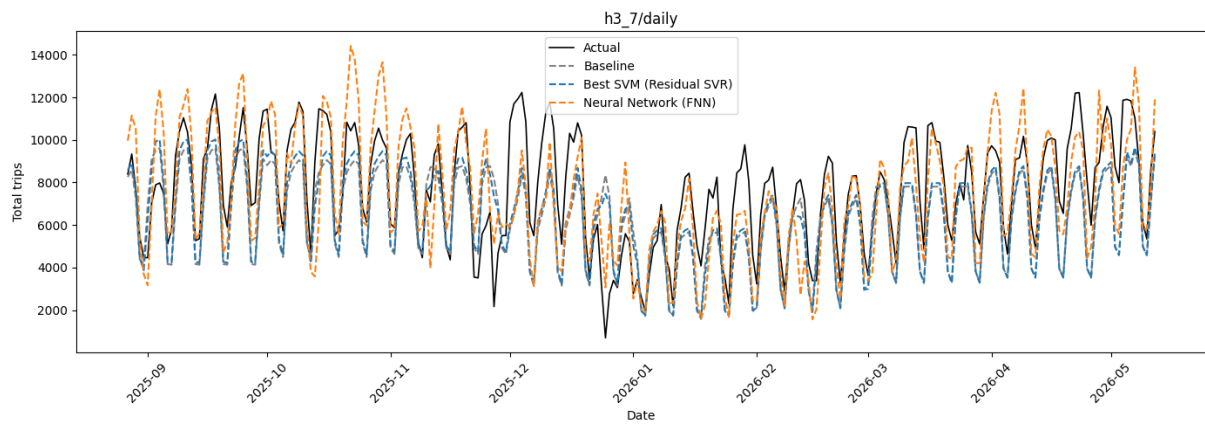
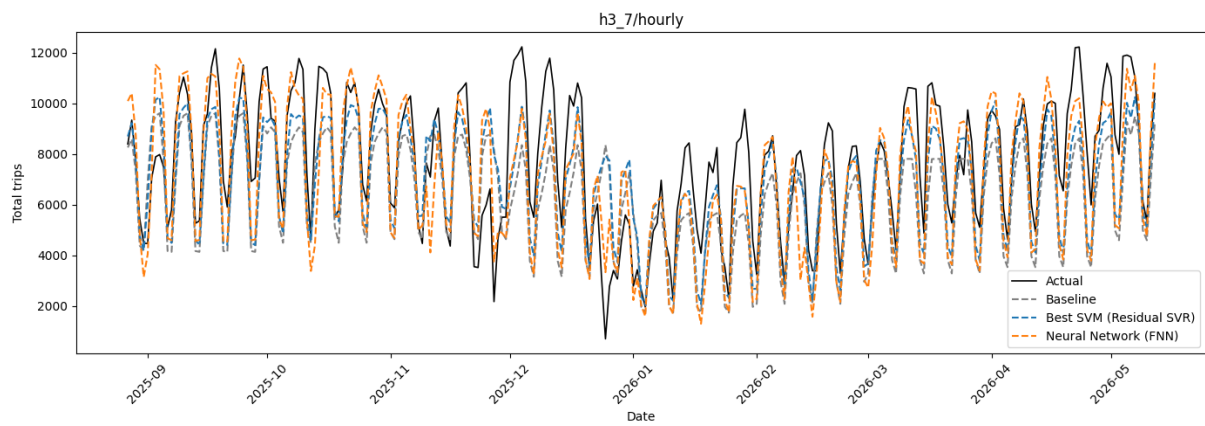


Figure 18: Test relative error (MAE / mean demand) by hour of day (h3_7/hourly).



(a) h3_7/daily



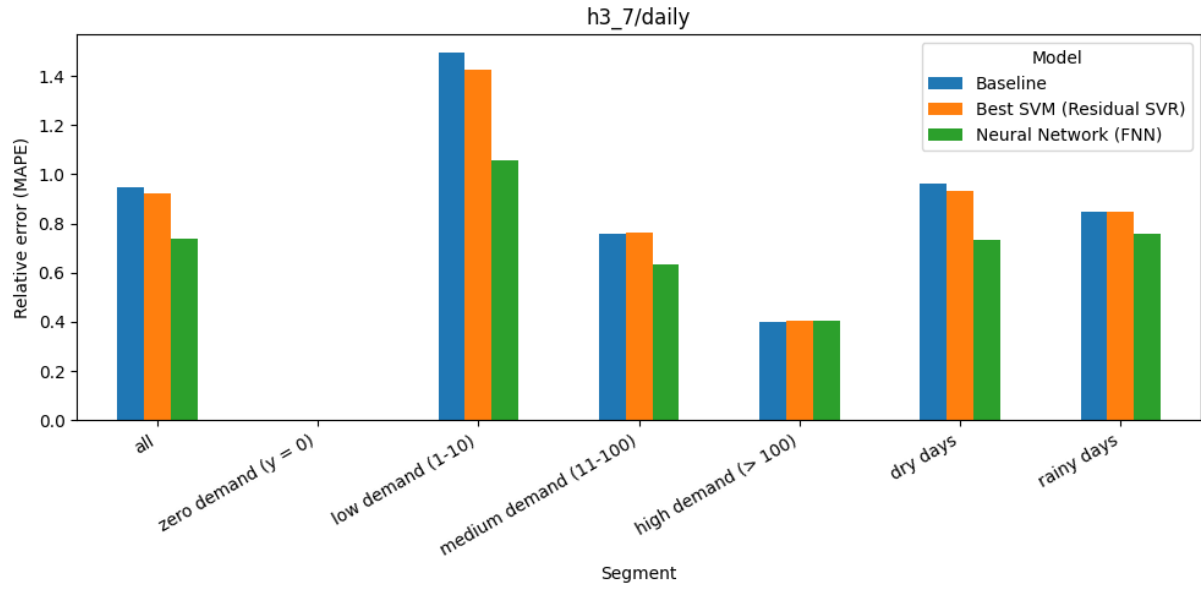
(b) h3_7/hourly

Figure 19: City-wide demand over the test period per temporal resolution.

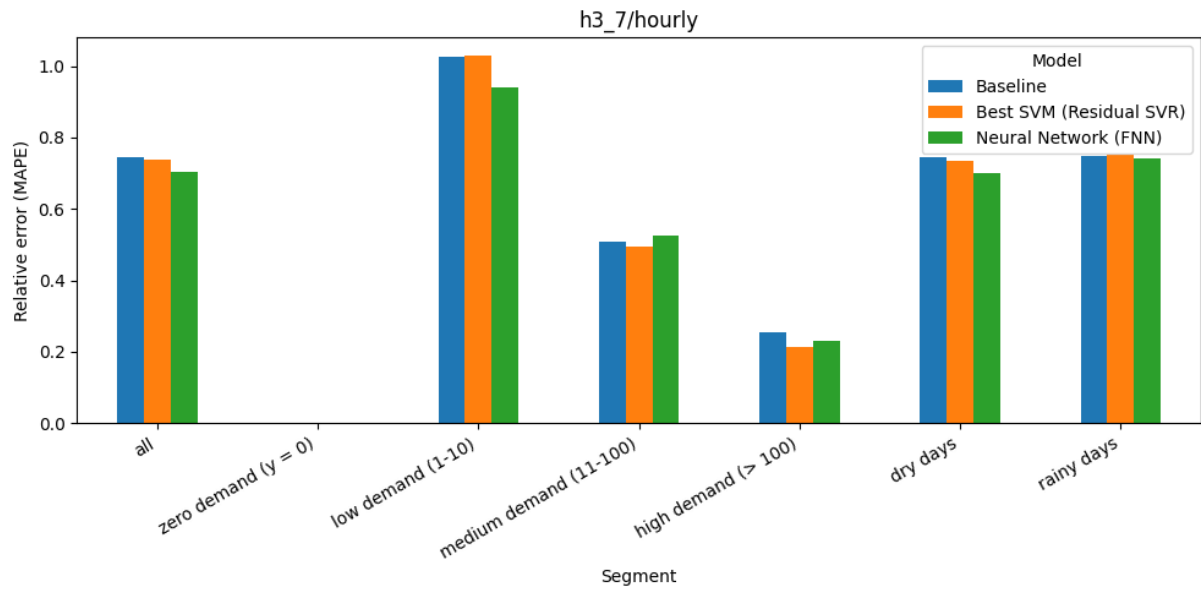
Table 9: Test MAE, relative error, and skill score by demand band and weather per temporal resolution.

(a) h3_7/daily		n	MAE	rel_error	skill_score
segment	Model				
all	Baseline	27972	21.391	0.948	0.000
	Best SVM (Residual SVR)	27972	20.639	0.923	0.035
	Neural Network (FNN)	27972	20.152	0.736	0.058
zero demand ($y = 0$)	Baseline	22149	0.126	NaN	0.000
	Best SVM (Residual SVR)	22149	0.189	NaN	-0.499
	Neural Network (FNN)	22149	0.188	NaN	-0.490
low demand (1-10)	Baseline	2384	4.149	1.494	0.000
	Best SVM (Residual SVR)	2384	3.974	1.427	0.042
	Neural Network (FNN)	2384	2.990	1.058	0.279
medium demand (11-100)	Baseline	1628	27.275	0.758	0.000
	Best SVM (Residual SVR)	1628	27.656	0.761	-0.014
	Neural Network (FNN)	1628	24.402	0.633	0.105
high demand (> 100)	Baseline	1811	298.878	0.401	0.000
	Best SVM (Residual SVR)	1811	286.379	0.404	0.042
	Neural Network (FNN)	1811	283.091	0.404	0.053
dry days	Baseline	24732	21.422	0.961	0.000
	Best SVM (Residual SVR)	24732	20.616	0.933	0.038
	Neural Network (FNN)	24732	19.679	0.733	0.081
rainy days	Baseline	3240	21.160	0.847	0.000
	Best SVM (Residual SVR)	3240	20.814	0.848	0.016
	Neural Network (FNN)	3240	23.766	0.760	-0.123

(b) h3_7/hourly		n	MAE	rel_error	skill_score
segment	Model				
all	Baseline	671328	1.068	0.746	0.000
	Best SVM (Residual SVR)	671328	1.016	0.738	0.048
	Neural Network (FNN)	671328	0.977	0.704	0.085
zero demand ($y = 0$)	Baseline	620830	0.043	NaN	0.000
	Best SVM (Residual SVR)	620830	0.107	NaN	-1.482
	Neural Network (FNN)	620830	0.031	NaN	0.277
low demand (1-10)	Baseline	26513	3.324	1.026	0.000
	Best SVM (Residual SVR)	26513	3.321	1.028	0.001
	Neural Network (FNN)	26513	3.171	0.939	0.046
medium demand (11-100)	Baseline	17335	15.334	0.507	0.000
	Best SVM (Residual SVR)	17335	14.823	0.494	0.033
	Neural Network (FNN)	17335	15.912	0.527	-0.038
high demand (> 100)	Baseline	6650	50.534	0.256	0.000
	Best SVM (Residual SVR)	6650	40.714	0.215	0.194
	Neural Network (FNN)	6650	41.552	0.231	0.178
dry days	Baseline	590544	1.058	0.746	0.000
	Best SVM (Residual SVR)	590544	1.004	0.736	0.051
	Neural Network (FNN)	590544	0.966	0.698	0.087
rainy days	Baseline	80784	1.143	0.749	0.000
	Best SVM (Residual SVR)	80784	1.110	0.752	0.028
	Neural Network (FNN)	80784	1.057	0.742	0.075



(a) h3_7/daily



(b) h3_7/hourly

Figure 20: Test relative error (MAE / mean demand) by segment per temporal resolution.

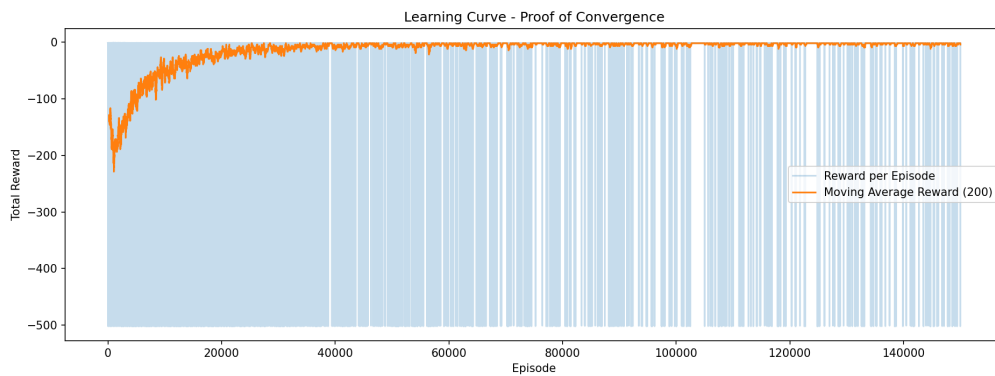


Figure 21: Figure 1a: Training score over time.

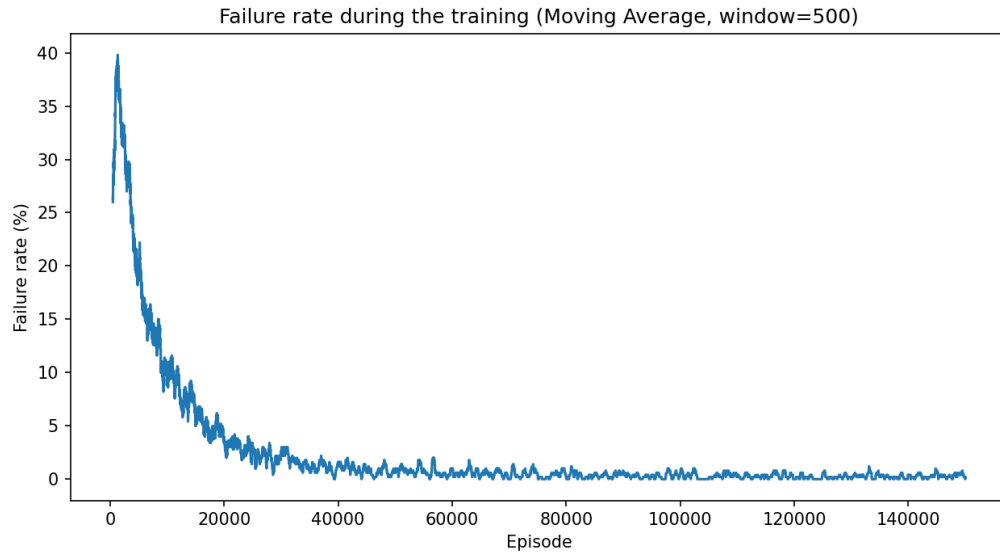


Figure 22: Figure 1b. Probability of running out of energy over time.

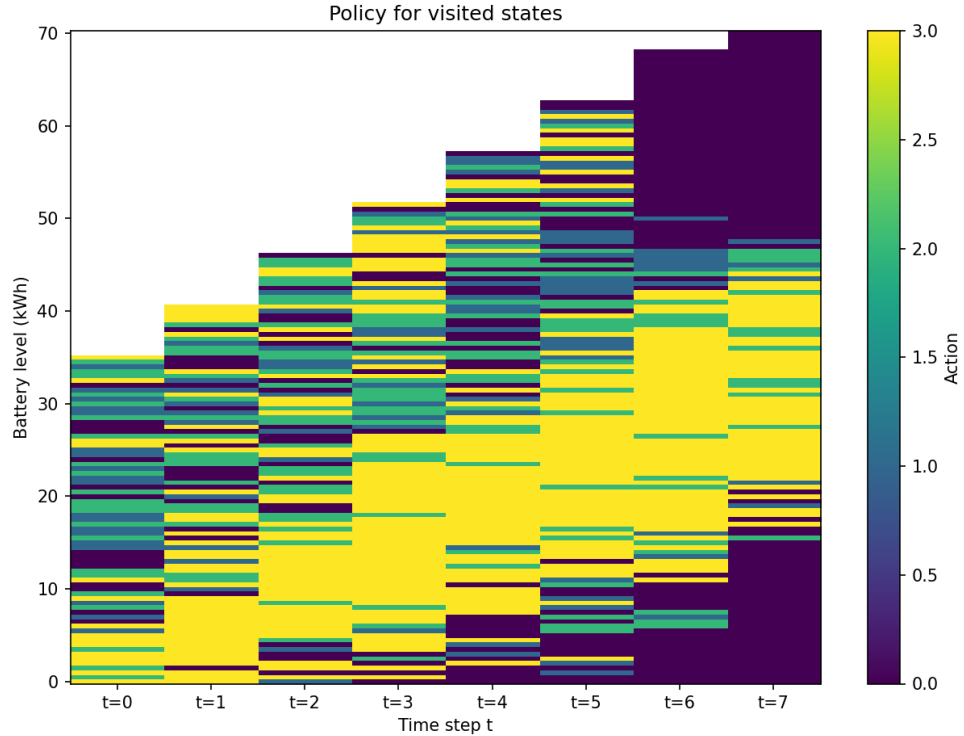


Figure 23: Figure 2. Learned charging policy. The color indicates the chosen charging speed for each battery-level/time combination.

Table 10: Table Naive vs. Tabular Q-Learning.

Strategy	Avg. Daily Cost	Probability of Running Out
0 Naive (always full power)	\$1.92	~0.14%
1 Tabular Q-Learning	\$1.50 $\pm$ \$0.02	0.11% $\pm$ 0.09%

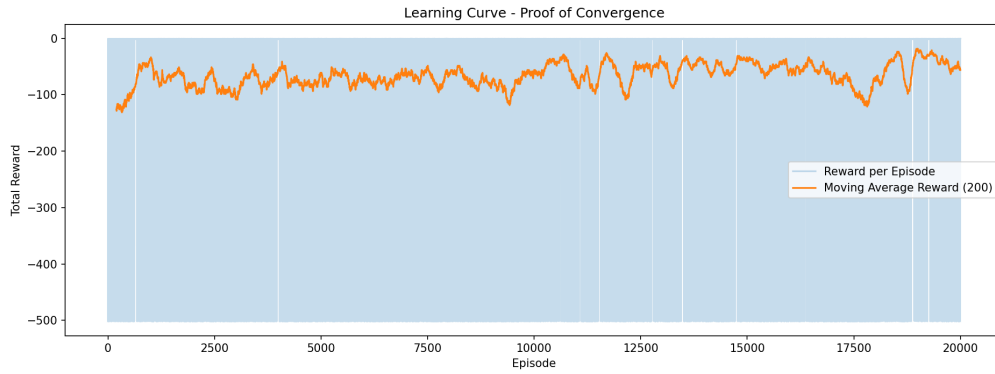


Figure 24: Figure 3a. DQN training score.

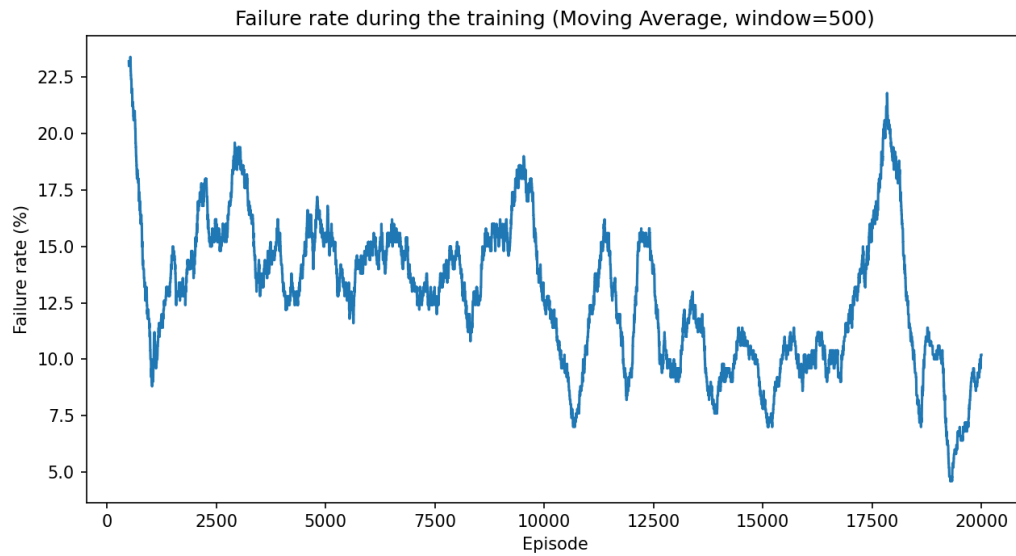


Figure 25: Figure 3b. DQN failure rate during training.

Table 11: Table DQN Summary.

Strategy	Avg. Daily Cost	Probability of Running Out
0 Deep Q-Network (DQN)	\$1.46 $\pm$ \$0.24	1.16% $\pm$ 0.95%

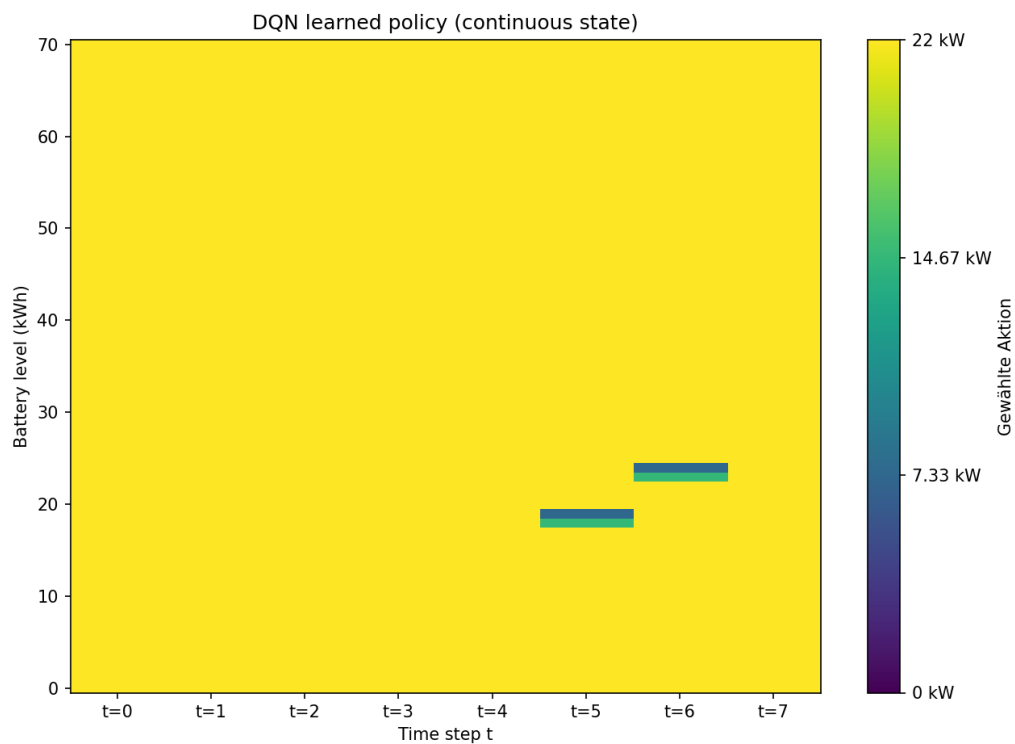


Figure 26: Figure 4. Best DQN policy found across the five training runs. The Q-table stores entries only for states that were actually visited during training (`visit_counts > 0`) and thus, combinations of time step and battery level that were never reached remain empty. The DQN network, by contrast, is a continuous function that produces an output for any given input. This includes states that never occurred during training (e.g. battery=60 at t=0). Thereby, both plots differ in terms of their appearance.

Table 12: Table DQN Per-Seed Breakdown.

	Run (seed)	Avg. Daily Cost	Probability of Running Out	Best checkpoint found at episode
0	0	\$1.29	2.82%	16,000
1	1	\$1.92	0.00%	8,000
2	2	\$1.28	1.48%	18,000
3	3	\$1.53	0.70%	12,000
4	4	\$1.31	0.78%	12,000

No additional labeled figures or tables from this notebook.

References

Yan, J., Xiang, L., Wu, C., & Wu, H. (2020). City-scale taxi demand prediction using multisource urban geospatial data. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 43, 213–220.